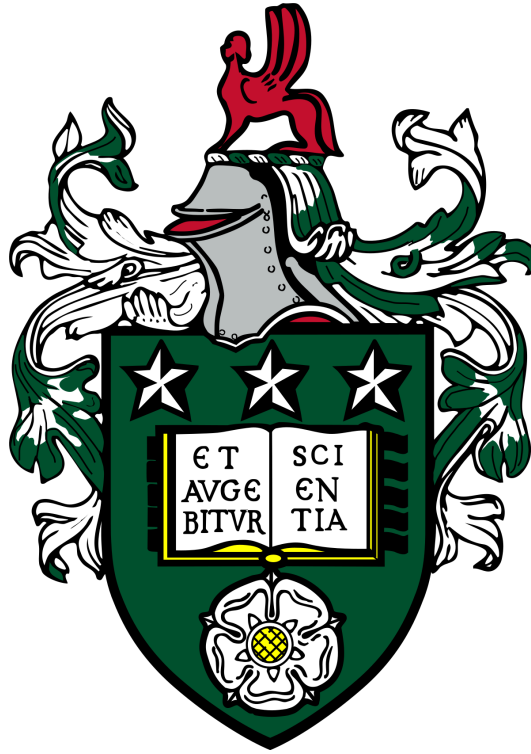


Modelling Polymer Flow



Robin Tester

Student Number: 201441256

Supervised by: Dr Claire McIlroy

MATH5000M

August 20, 2025

In this dissertation we study the mathematics of polymer melts, a non Newtonian fluid. To start, we introduce polymer chains and how to model them mathematically, following theory presented by Doi & Edwards [1]. We state some properties of a single polymer chain such as relaxation time and radius of gyration. We then introduce two models for polymer chains: the dumbbell and tube models. The dumbbell model is the simplest model that incorporates the elasticity of a single polymer chain and the tube model is used to handle relaxation of entanglements between multiple polymer chains. We derive a deformation equation for the dumbbell model. Following Likhtman & Graham [2], we extend the tube model to the Rolie-Poly constitutive model that includes more detailed descriptions of relaxation. We also derive a deformation equation for the Rolie-Poly model. We subject both the dumbbell and Rolie-Poly model to shear flows and discuss the flow profile, stress profile and stretch profile.

We then introduce polydisperse polymers and, following the work of Boudara [3], discuss a constitutive model for a bi-disperse polymer melt, based on the Rolie-Poly model. We subject this to a shear flow and discuss the effect of changing the composition of the polymer melt.

Finally, we derive the flow profile for a Newtonian fluid before subjecting a monodisperse polymer melt to a pipe flow. We solve for the flow profile of a melt pipe flow, following the work of McIlroy & Olmsted, to compare it to that of a Newtonian fluid. We justify the differences by investigating the stretch and stress profiles.

Contents

1	Introduction	4
2	Polymer Modelling	5
2.1	Stress	6
2.2	Polymer chains	7
2.3	Dumbbell Model	9
2.3.1	Resting state of a dumbbell	11
2.3.2	Shear Flow	12
2.4	Tube model	14
2.4.1	Reptation vs relaxation	15
2.5	Rolie-Poly model	15
2.5.1	Shear flow	17
2.5.2	Start up behaviour	19
3	Modelling a polydisperse polymer melt	24
3.1	Polydisperse polymers	24
3.2	Rolie-Double-Poly model	26
3.3	Shear flow	28
3.4	Comparison to a monodisperse melt	29
4	Viscous flow through a pipe	30
4.1	Newtonian Flows	30
4.1.1	Basic steady state flow	30
4.1.2	Full time solution	32
4.2	Polymer Flows	33
4.2.1	Dumbbell model in pipe	35
4.2.2	Tube model in pipe flow	37
5	Conclusion	40
6	Appendix	41
	Bibliography	56

1 Introduction

Polymers can be found everywhere in everyday life. They can be natural, such as DNA or protein chains in cells, or synthetic such as polystyrene. Synthetic chains are man-made and used in consumer products all over the globe (in raincoats or packing peanuts, for example). Polymers are long molecules comprised of lots of repeating units, monomers, that are linked together with strong covalent bonds. Due to the high strength of these covalent bonds, most polymers are solid at room temperature.

Polymers in a liquid form are known as polymer melts and they contain thousands of polymer chains. Melts are usually quite viscous and elastic. For example, chewing gum is a polymer melt that can be stretched out and shrunk. Investigating this viscoelasticity property is one of the main interests of Rheology.

Polymers are used in so many things because of the versatile range of properties they can have. They often have a lower density than other solid materials and have high extensibility so can be stretched as necessary. Despite their lightweight composition, they are often strong. Another attractive property of polymers are low melting points. Even though they share these main properties, different polymer chains will have different lengths and different molecular weights. Different monomers have different weights, thus the weight of the polymer chain depends on which monomers are in its composition.

Industry-used polymer melts are typically polydisperse: they are composed of polymers that have a whole distribution of chains lengths and molecular weights. Particular cases of this are mono-disperse and bi-disperse melts. Mono-disperse melts have only one kind of polymer within; that is, chains with an equal length and molecular weight. Bi-disperse have two different kinds of polymers within.

The chains within a melt move randomly and can become entangled with each other, much like a bowl of spaghetti or cables in a pocket. Because of these entanglements, it becomes hard to describe the dynamics of a single polymer chain within the melt and assumptions must be made about the orientation of

the polymers to create constitutive models.

Due to the viscoelastic properties and entanglements, one cannot describe flows of polymer melts in the same way as a Newtonian fluid. As we will see, there is an extra contribution to the stress tensor [4] so the Navier-Stokes equations cannot be used. As a polymer can be stretched and deformed, it can be harder to track the movement of a single chain compared to a single fluid particle in traditional Newtonian fluid dynamics.

One of the largest applications of polymer melts is in 3D printing, in which consumer grade plastic goods can be created without the use of a mould, even in cases where non-trivial shapes must be created. The process is seemingly simple: a polymer melt is extruded through a nozzle to build up layers of plastic on a plate in the $x - y$ plane. Once one layer has been printed, the nozzle moves slightly higher in the z direction to build the next layer, slowly building a print layer by layer. Polymer dynamics are so integral for 3D printing because the melts must be transported from a reserve to the nozzle. Thus, we must know how they flow in order to create effective methods of transport.

In this project, we will give a brief introduction to rheology and the mathematics of polymer chains by considering different models. In particular, we will discuss these models in the context of a shear flow. We will then discuss pipe flow for Newtonian and polymer flows and investigate the differences.

2 Polymer Modelling

In this section we will introduce some important concepts from rheology that will form the basis of this dissertation. We will follow theory presented by de Gennes [5], Doi and Edwards [1] and Boudara [3]. Rheology is defined as the study of materials - in particular, the mechanical properties of those materials. These properties include the stress and strain of a material and what impacts they have on the deformation of the material.

2.1 Stress

In this section we will define the stress tensor for a *Newtonian* fluid and the discuss what happens to the stress tensor in a polymer solution.

The deformation rate, or deformation tensor, of a fluid is calculated from the velocity field, $\mathbf{u}(\mathbf{r}, t)$, and is denoted by the velocity gradient tensor,

$$\boldsymbol{\kappa}(\mathbf{r}, t) = (\nabla \mathbf{u})^T. \quad (2.1)$$

Equation (2.1) is commonly split into two parts: a *symmetric* rate of strain tensor, $\mathbf{E}$, and a *traceless* vorticity tensor, $\mathbf{\Omega}$, with

$$\begin{aligned} \mathbf{E} &= \frac{1}{2}(\boldsymbol{\kappa} + \boldsymbol{\kappa}^T), \\ \mathbf{\Omega} &= \frac{1}{2}(\boldsymbol{\kappa} - \boldsymbol{\kappa}^T). \end{aligned}$$

Thus, the deformation tensor can be expressed as

$$\boldsymbol{\kappa} = \mathbf{E} + \mathbf{\Omega}. \quad (2.2)$$

In a Newtonian fluid (which has a linear relation between stress rate and local strain), the full stress tensor is

$$\boldsymbol{\sigma} = p\mathbf{I} + 2\mu(\nabla \mathbf{u} + (\nabla \mathbf{u})^T) = 2\mu\mathbf{E}. \quad (2.3)$$

where μ is the fluid viscosity and p is the pressure acting on the fluid. However, in a melted polymer solution there is a non-linear relationship between stress and strain which leads to the following stress tensor

$$\boldsymbol{\sigma} = p\mathbf{I} + 2\mu\mathbf{E} + G_e(\mathbf{A} - \mathbf{I}). \quad (2.4)$$

Here we have an extra contribution to the stress tensor. The final term is the plateau modulus, G_e , multiplied by a polymer deformation tensor, $\mathbf{A}$. The deformation tensor takes the form

$$\mathbf{A} = \begin{pmatrix} A_{xx} & A_{xy} & A_{xz} \\ A_{yx} & A_{yy} & A_{yz} \\ A_{zx} & A_{zy} & A_{zz} \end{pmatrix}. \quad (2.5)$$

The extra term causes issues when solving flow problems because the Navier-Stokes equations cannot be used. Any problem must be derived from the general form of the momentum equation.

2.2 Polymer chains

We can consider a polymer chain mathematically as N points connected by springs of length b , giving $N - 1$ segments of a chain. This is the Rouse model, first proposed in 1953 by Rouse [6]. He proposed that the chain can diffuse by Brownian motion and every particle in the chain exerts a force on both of its neighbours. Each segment in the chain can be modelled as an elastic dumbbell [7], as seen in Figure 2. The “dumbbell” consists of two points connected by a spring with spring constant k (as seen in Figure 1). This type of model is the simplest way to incorporate elasticity. The spring stretches and relaxes on some time scale that we can find.

The chain configuration is represented formally by the set of position vectors of the points $\{\mathbf{R}_N\} = (\mathbf{R}_0, \dots, \mathbf{R}_N)$. Then the total length of the polymer is represented by the *end to end* vector

$$\mathbf{R} = \mathbf{R}_N - \mathbf{R}_0. \quad (2.6)$$

Since each segment of the chain follows a Gaussian distribution, they can be treated as independent from one other. We have that the mean length of a freely-jointed polymer is

$$\langle \mathbf{R} \rangle = 0 \quad (2.7)$$

where $\langle \cdot \rangle$ denotes the average. The variance of a polymer chain is given by

$$\langle \mathbf{R}^2 \rangle = Nb^2. \quad (2.8)$$

The distribution of the polymer length is given by

$$p(N, \mathbf{r}) = \left(\frac{3}{2\pi Nb^2} \right)^{3/2} \exp \left(-\frac{3\mathbf{R}^2}{2Nb^2} \right). \quad (2.9)$$

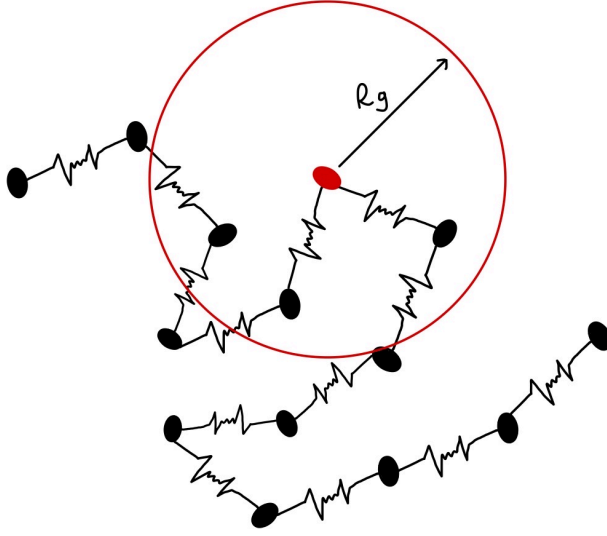


Figure 1: Rouse model of polymer chains with length b and radius of gyration R_g .

Since all of the particles in a chain are connected, if one moved by a distance R , say, it would have to pull on all the particles within a circle of radius R . This circle is known as the radius of gyration and is denoted R_g . We can find the value of R_g for an ideal polymer. Starting from the definition of R_g [8],

$$\begin{aligned}
 R_g^2 &= \frac{1}{2N^2} \sum_{i=1}^N \sum_{j=1}^N \langle (r_i - r_j)^2 \rangle \\
 &= \frac{b^2}{2N^2} \sum_{i=1}^N \sum_{j=1}^i i^2 - j \\
 &= \frac{b^2}{2N^2} \left(\frac{N(N+1)(2N+1)}{6} - \frac{N(N+1)}{2} \right) = \frac{b^2}{2N^2} \left(\frac{N^3}{3} + O(N^2) \right) \\
 &= \frac{Nb^2}{6} \Rightarrow R_g = \frac{1}{\sqrt{6}} \sqrt{N} a^2.
 \end{aligned}$$

The polymer deformation tensor can be expressed in terms of the end-to-end vector and radius of gyration,

$$A_{ij} = \frac{\langle R_i R_j \rangle}{3R_g^2}. \quad (2.10)$$

Expressing the tensor in this way is useful for deriving equations of motion for the deformation, as we will see in the next section. Furthermore, the stretching of a polymer can be described by the trace of $\mathbf{A}$. When $i = j$, we have

$$A_{ii} = \frac{\langle |R_i|^2 \rangle}{3R_g^2}$$

Which is a measure of distance from equilibrium. Thus we denote the stretch of the polymer chain by $\Lambda = \text{tr} \mathbf{A}$. Often we will normalise this by setting $\Lambda = 3/\text{tr} \mathbf{A}$.

The time scale in which the springs stretch and relax has been well studied. Here, we will just state the main result from Doi and Edwards, but for more detail see chapter 4 of [1]. The longest relaxation time for an N particle polymer chain is

$$\tau_R = \frac{\zeta N^2 b^2}{3\pi^2 k_B T} \quad (2.11)$$

Where T is the temperature of the polymer and k_B is the Boltzmann constant. For our purposes, relaxation time can be related to the molecular weight of a considered polymer. From Equation (2.8) we have $R_g \sim \sqrt{Nb^2} \sim M_w^{\frac{1}{2}}$ and we also have that diffusion coefficient scales like $D \sim M_w^{-1}$. So the relaxation time scales like

$$\tau_R \sim \frac{R_g^2}{D} \sim \frac{M_w}{M_w^{-1}} = M_w^2. \quad (2.12)$$

2.3 Dumbbell Model

We will now consider a single segment from a polymer chain. That is, two particles connected by a spring with spring constant k . We aim to study how this single chain deforms within a melt. Here, we derive an equation for this deformation.

The dynamics of the model can be found by considering the forces acting on the dumbbell when it is in a flow. There are 3 forces: a relaxation force by the spring, a drag force acting on the molecules due to the movement of the flow and a thermal force due to random collisions of the molecules with the surrounding particles.

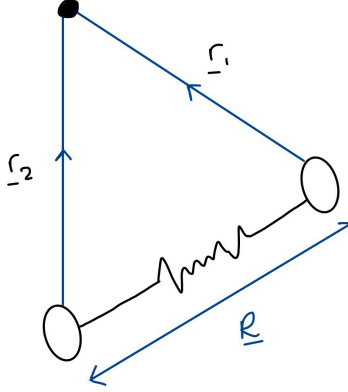


Figure 2: A polymer modelled as a spring, of length $\mathbf{R}$, between two points.

The resultant force on both of the particles is

$$\mathbf{F}_{\text{spring}} + \mathbf{F}_{\text{drag}} + \mathbf{F}_{\text{thermal}} = 0.$$

From here we can get 2 equations of motion for the particles. For particle 1:

$$k(\mathbf{r}_1 - \mathbf{r}_2) - \zeta \left(\frac{d\mathbf{r}_1}{dt} - \mathbf{u}(\mathbf{r}_1) \right) + \mathbf{f}_1 = 0, \quad (2.13)$$

and similarly for particle 2:

$$k(\mathbf{r}_2 - \mathbf{r}_1) - \zeta \left(\frac{d\mathbf{r}_2}{dt} - \mathbf{u}(\mathbf{r}_2) \right) + \mathbf{f}_2 = 0. \quad (2.14)$$

where ζ is the coefficient of drag and $\mathbf{f}_1, \mathbf{f}_2$ are thermal forces. By subtracting Equation (2.13) from Equation (2.14) and recalling $\mathbf{R} = \mathbf{r}_2 - \mathbf{r}_1$ we have

$$\zeta \frac{d\mathbf{R}}{dt} - \zeta(\mathbf{u}(\mathbf{r}_2) - \mathbf{u}(\mathbf{r}_1)) = 2k\mathbf{R} + \mathbf{f}. \quad (2.15)$$

We can expand $\mathbf{u}$ around $\mathbf{r}_1$ with a Taylor expansion, giving

$$u_i(\mathbf{r}_2) = u_i(\mathbf{r}_1) + \frac{\partial u_i}{\partial x_k} (\mathbf{r}_2 - \mathbf{r}_1)_k = u_i(\mathbf{r}_1) + \frac{\partial u_i}{\partial x_k} R_k.$$

Substitution into Equation (2.15) gives

$$\zeta \frac{d\mathbf{R}}{dt} - \zeta(\nabla \mathbf{u} \cdot \mathbf{R}) = 2k\mathbf{R} + \mathbf{f}, \quad (2.16)$$

which in tensor notation is

$$\frac{dR_i}{dt} - \nabla_k u_i R_k - \frac{2k}{\zeta} R_i + \frac{f_i}{\zeta}. \quad (2.17)$$

We want an equation in terms of $A_{ij} = \langle R_i R_j \rangle$, so multiplying by R_j gives

$$R_j \frac{dR_i}{dt} = \nabla_k u_i R_k R_j - \frac{2k}{\zeta} R_i R_j + \frac{f_i}{\zeta} R_j. \quad (2.18)$$

Then we can get another equation in terms of R_i and R_j by cyclic permutation of Equation (2.18), giving

$$R_i \frac{dR_j}{dt} = \nabla_k u_j R_i - \frac{2k}{\zeta} R_j R_i + \frac{f_j}{\zeta} R_i. \quad (2.19)$$

Adding these together yields

$$\frac{d}{dt}(R_i R_j) = \nabla_k u_i R_k R_j + R_i R_k (\nabla_i u_k)^T - \frac{4k}{\zeta} R_i R_j + \frac{1}{\zeta} (f_i R_j + (f_i R_j)^T). \quad (2.20)$$

Taking the average gives

$$\frac{d}{dt} \langle R_i R_j \rangle = \nabla_k u_i \langle R_k R_j \rangle + \langle R_i R_k \rangle (\nabla_j u_k)^T - \frac{4k}{\zeta} \langle R_i R_j \rangle + \frac{4k_B T}{\zeta} \delta_{ij}. \quad (2.21)$$

Letting $Q_{ij} = \langle R_i R_j \rangle$ gives

$$\frac{d\mathbf{Q}}{dt} = \nabla \mathbf{u} \cdot \mathbf{Q} + \mathbf{Q} \cdot (\nabla \mathbf{u})^T - \frac{4k}{\zeta} \mathbf{Q} + \frac{4k_B T}{\zeta} \mathbf{I}. \quad (2.22)$$

Finally, introducing a time scale $\tau = \zeta/(4k)$ and a re-scaling $\mathbf{A} = 3\mathbf{Q}/Nb^2$ gives

$$\frac{d\mathbf{A}}{dt} = \nabla \mathbf{u} \cdot \mathbf{A} + \mathbf{A} \cdot (\nabla \mathbf{u})^T - \frac{1}{\tau} (\mathbf{A} - \mathbf{I}). \quad (2.23)$$

2.3.1 Resting state of a dumbbell

The simplest case we can study is to put the dumbbell into a system with no motion ($\mathbf{u} = \mathbf{0}$). In this scenario, $\nabla \mathbf{u} = (\nabla \mathbf{u})^T = 0$, so Equation (2.23) reduces to

$$\frac{d\mathbf{A}}{dt} = -\frac{1}{\tau} (\mathbf{A} - \mathbf{I}) \Rightarrow \mathbf{A} = \mathbf{I} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}. \quad (2.24)$$

Physically, this means that the dumbbell will not deform at all if there is no flow in the system, regardless of how long it has been sitting there. This is exactly what is to be expected. There are no drag forces acting on the dumbbell, and since there is no motion in its surroundings, there will be no forces due to collisions. Thus, by Newton's first law [9], the dumbbell will never deform.

2.3.2 Shear Flow

Now we will introduce a simple flow into the system. Consider a steady state shear flow $\mathbf{u} = (\gamma(y), 0, 0)$. This gives the velocity gradient tensor

$$\nabla \mathbf{u} = \frac{\partial u_i}{\partial x_j} = \begin{pmatrix} 0 & \dot{\gamma} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \Rightarrow (\nabla \mathbf{u})^T = \begin{pmatrix} 0 & 0 & 0 \\ \dot{\gamma} & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}. \quad (2.25)$$

Substituting into Equation (2.23) yields

$$\begin{pmatrix} \dot{\gamma} A_{xx} & \dot{\gamma} A_{yy} & \dot{\gamma} A_{yz} \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} + \begin{pmatrix} \dot{\gamma} A_{xy} & 0 & 0 \\ \dot{\gamma} A_{yx} & 0 & 0 \\ \dot{\gamma} A_{zy} & 0 & 0 \end{pmatrix} = \frac{1}{\tau_d} \begin{pmatrix} A_{xx} - 1 & A_{xy} & A_{xz} \\ A_{yx} & A_{yy} - 1 & A_{yz} \\ A_{zx} & A_{zy} & A_{zz} - 1 \end{pmatrix} \quad (2.26)$$

Now by equating components and solving the linear system of equations we have an explicit form of the polymer deformation tensor

$$\mathbf{A} = \begin{pmatrix} 1 + 2\dot{\gamma}^2 \tau_d^2 & \dot{\gamma} \tau_d & 0 \\ \dot{\gamma} \tau_d & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}. \quad (2.27)$$

The formation of the polymer in equilibrium can be seen in Figure 3; the deformation tensor has no effect on the orientation of the dumbbell. Similarly, the formation of the polymer in a shear flow can be seen in Figure 4; this time the deformation tensor acts on the dumbbell which deforms it such that R_g is ellipsoidal. This deformation makes intuitive sense because it matches the profile for a shear flow (as seen in Figure 5).

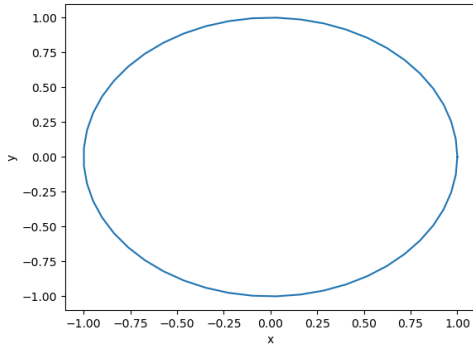


Figure 3: Polymer solution in an equilibrium

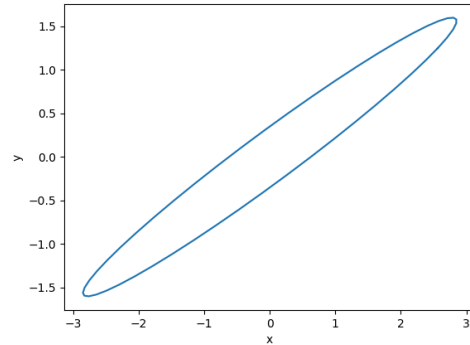


Figure 4: Polymer solution subject to a simple steady state shear flow

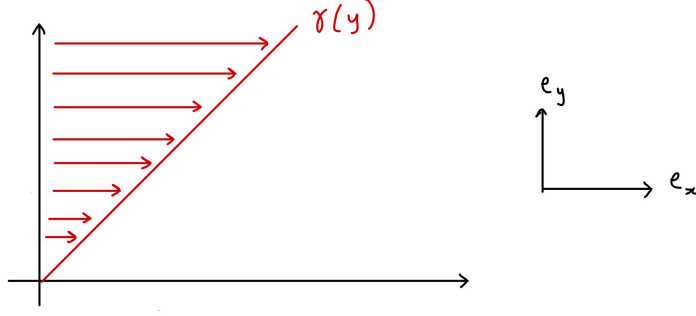


Figure 5: Shear flow

Equation (2.23) can be solved when the flow is not in steady state, by method of separation of variables and integrating factors. The method is the similar for all 9 components so we will only do the A_{xy} component and then state the full solution. We have

$$\frac{dA_{xy}}{dt} = \dot{\gamma} - \frac{1}{\tau_d} A_{xy}, \quad A_{xy}(0) = 0. \quad (2.28)$$

Introducing an integrating factor of e^{t/τ_d} and rearranging gives

$$\begin{aligned} \frac{d}{dt}(A_{xy}e^{t/\tau_d}) &= \dot{\gamma}e^{t/\tau_d} \\ \Rightarrow A_{xy}e^{t/\tau_d} &= \tau_d \dot{\gamma} + C. \end{aligned} \quad (2.29)$$

Then by the initial condition we have $C = -\tau_d \dot{\gamma}$,

$$\Rightarrow A_{xy} = \tau_d \dot{\gamma} - \tau_d \dot{\gamma} e^{-t/\tau_d}. \quad (2.30)$$

The full solution is

$$\mathbf{A}(t) = \begin{pmatrix} A_{xx} & A_{xy} & 0 \\ A_{xy} & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad (2.31)$$

with

$$\begin{aligned} A_{xx} &= 1 + 2\dot{\gamma}^2 \tau_d^2 (1 - e^{-t/\tau_d}) - 2\dot{\gamma}^2 \tau_d t e^{-t/\tau_d}, \\ A_{xy} &= \dot{\gamma} \tau_d (1 - e^{-t/\tau_d}). \end{aligned}$$

If we let $t \rightarrow \infty$, we recover the steady state solution.

2.4 Tube model

The dumbbell model considered the deformation of a melt with only a single polymer chain. When more than one chain is considered, problems arise due to the chains entangling with one another, much like a bowl of spaghetti or cables in a pocket. This entanglement between the chains acts to slow down the movement of a single polymer. One way to deal with this problem, mathematically, is to consider a polymer chain inside of a “tube” running parallel to the polymer’s contour, as first done by Doi & Edwards [1]. By constricting the entangled polymers inside a tube, as seen in Figure 6, the movement of the polymers is restricted to the direction of their own contours. Subsequently, there is no movement perpendicular to the direction of the tube and so no entanglements across chains.

Much like the dumbbell’s relaxation time, the tube has a unique relaxation mechanism. As there are now multiple polymers in the melt, movement along the tube is caused by random collisions of the tube and its neighbouring particles. Movement due to these collisions is known as *reptation*. The tube model incorporates both reptation and relaxation; the polymer chain inside the tube still relaxes according to Equation (2.12) and the tube will move due to the reptation.

Since the tube is modelled from a polymer chain, it’s length still follows the same Gaussian distribution as a single polymer chain, so the time it takes for a tube to move distance L , equal to its length is the *reptation time*, denoted τ_d , which scales like

$$\tau_d \sim \frac{L^2}{D} \sim \frac{M_w^2}{M_w^{-1}} = M_w^3.$$

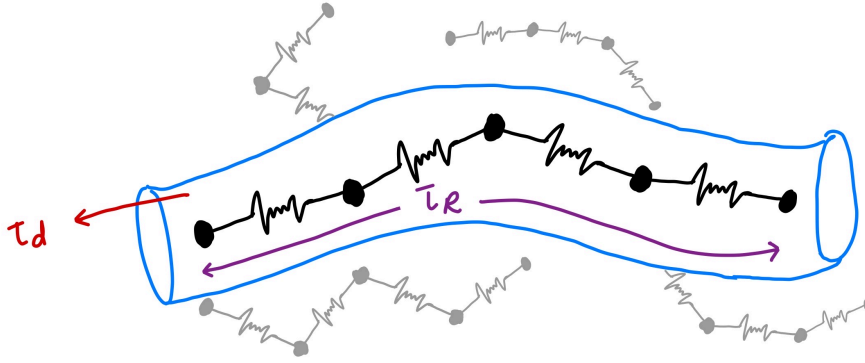


Figure 6: A polymer chain (black) constrained in a tube (blue) entangled by other polymer chains (grey) showing two types of diffusion: relaxation time (τ_R) and reptation time (τ_d).

While the tube model is reliable for modelling mono-disperse polymer melts, problems arise when considering melts of polydisperse polymers. However, the theory can serve as a baseline for these melts, as we will see in Section 3.

The tube model also fails to explain some experimental results, such as the stress at high shear rates [10]. According to the model decreases with increasing shear rates but experiments show is actually increases with an increasing shear rate. These results need extra consideration when part of a full model, as we will see in Section 2.5.

2.4.1 Reptation vs relaxation

It is useful to emphasize the difference between the two relaxation methods. The Rouse time, denoted τ_r , describes the time it takes for a whole polymer chain to stretch and relax. However, the reptation time, τ_d , is unique to the tube model and describes how long it takes for the whole tube (whose interior contains polymer chains) to stretch and relax.

2.5 Rolie-Poly model

A particular example of a tube model is the Rolie-Poly model [2], which stands for ROuse, LInear Entangled POLYmers. This model accounts for the discrepancy from experimental data. A relaxation mechanism known as Convective Constraint Release (CCR), first proposed by Marrucci [10] is introduced. At large shear

strength, the flow causes the polymer chains to move further away from the neighbouring chains, so each polymer has more room to diffuse out by reptation. There is effectively a relaxation method induced by the flow, and the Rolie-Poly model includes a term that accounts for this relaxation.

The derivation of the equation, found in [11], involves some in depth thermodynamics, so we will not go into much detail as that is beyond the scope of this dissertation. We will however, outline the second half of the argument which highlights how the stress behaves at high and low stretch. The main result is a differential equation in $\boldsymbol{\sigma}$ that describes the stress over time:

$$\frac{d\boldsymbol{\sigma}}{dt} = \nabla \mathbf{u} \cdot \boldsymbol{\sigma} + \boldsymbol{\sigma} \cdot (\nabla \mathbf{u})^T - \frac{1}{\tau_d}(\boldsymbol{\sigma} - \mathbf{I}) - f_{retr}(\text{tr}\boldsymbol{\sigma})\boldsymbol{\sigma} - f_{CCR}(\text{tr}\boldsymbol{\sigma})(\boldsymbol{\sigma} - \mathbf{I}) \quad (2.32)$$

Where f_{retr} describes relaxation and f_{CCR} describes CCR. These functions are determined through physical arguments. In the case of small stretch ($\text{tr}\boldsymbol{\sigma} - 3 \ll 1$), the relaxation function must be proportional to the stretch as the tube must be allowed to relax to its equilibrium state ($\text{tr}\boldsymbol{\sigma} = 3$) and since the typical relaxation time is τ_r , we have $f_{retr} \sim 2(\text{tr}\boldsymbol{\sigma} - 3)/\tau_R$. The CCR mechanism must be proportional to the relaxation (as at zero shear there is no CCR) so we have $f_{CCR} = 2\beta(\text{tr}\boldsymbol{\sigma} - 3)/\tau_R$. Here, β is a real number that describes the amount of CCR. A low β means less relaxation due to CCR.

At high stretch ($\text{tr}\boldsymbol{\sigma} - 3 \gg 1$), the stretch must follow the relation $d\lambda/dt = -(\lambda - 1)/\tau_R$, as seen in [12]. So with $\lambda = \sqrt{\text{tr}\boldsymbol{\sigma}/3}$, we must have $f_{retr} = 2/\tau_R$ as $\text{tr}\boldsymbol{\sigma} \rightarrow \infty$ to satisfy this requirement. The CCR function at high stretch will depend on the stretch, so we take $f_{CCR} = 2\beta(\sqrt{\text{tr}\boldsymbol{\sigma}/3})/\tau_R$. Thus, we have:

$$f_{retr} = \begin{cases} \frac{2(\text{tr}\boldsymbol{\sigma}-3)}{\tau_R} & \text{for } \text{tr}\boldsymbol{\sigma} - 3 \ll 1, \\ \frac{2}{\tau_R} & \text{for } \text{tr}\boldsymbol{\sigma} - 3 \gg 1, \end{cases}$$

and

$$f_{CCR} = \begin{cases} \frac{2\beta(\text{tr}\boldsymbol{\sigma}-3)}{\tau_R} & \text{for } \text{tr}\boldsymbol{\sigma} - 3 \ll 1, \\ \frac{2\beta\sqrt{(\text{tr}\boldsymbol{\sigma}/3)}}{\tau_R} & \text{for } \text{tr}\boldsymbol{\sigma} - 3 \gg 1. \end{cases}$$

Then, these functions must still have a value when switching between the two behaviours, so we can choose

$$f_{retr} = \frac{2\left(1 - \sqrt{\frac{3}{\text{tr}\boldsymbol{\sigma}}}\right)}{\tau_R}, \quad f_{CCR} = \frac{2\beta(\text{tr}\boldsymbol{\sigma})\left(1 - \sqrt{\frac{3}{\text{tr}\boldsymbol{\sigma}}}\right)}{\tau_R}.$$

Substituting these into Equation (2.32) gives the Rolie-Poly equation:

$$\begin{aligned} \frac{d\mathbf{A}}{dt} = & \nabla \mathbf{u} \cdot \mathbf{A} + \mathbf{A} \cdot (\nabla \mathbf{u})^T - \frac{1}{\tau_d}(\mathbf{A} - \mathbf{I}) \\ & - \frac{2}{\tau_R} \left(1 - \sqrt{\frac{3}{\text{tr} \mathbf{A}}}\right) \left(\mathbf{A} + \beta \sqrt{\frac{3}{\text{tr} \mathbf{A}}}(\mathbf{A} - \mathbf{I})\right). \end{aligned} \quad (2.33)$$

Where we have swapped $\boldsymbol{\sigma}$ for $\mathbf{A}$ due to the form of our stress tensor (Equation (2.4)).

2.5.1 Shear flow

First we will solve the Rolie-Poly equation in a steady state so we are to solve

$$\begin{aligned} 0 = & \nabla \mathbf{u} \cdot \mathbf{A} + \mathbf{A} \cdot (\nabla \mathbf{u})^T - \frac{1}{\tau_d}(\mathbf{A} - \mathbf{I}) - \\ & \frac{2}{\tau_R} \left(1 - \sqrt{\frac{3}{\text{tr} \mathbf{A}}}\right) \left(\mathbf{A} + \beta \sqrt{\frac{3}{\text{tr} \mathbf{A}}}(\mathbf{A} - \mathbf{I})\right). \end{aligned} \quad (2.34)$$

Substituting the velocity tensor for a shear flow gives

$$\begin{aligned} 0 = & \begin{pmatrix} \dot{\gamma}(A_{xy} + A_{yx}) & \dot{\gamma}A_{yy} & \dot{\gamma}A_{yz} \\ \dot{\gamma}A_{yy} & 0 & 0 \\ \dot{\gamma}A_{zy} & 0 & 0 \end{pmatrix} - \frac{1}{\tau_d}(\mathbf{A} - \mathbf{I}) \\ & - \frac{2}{\tau_R} \left(1 - \sqrt{\frac{3}{\text{tr} \mathbf{A}}}\right) \left(\mathbf{A} + \beta \sqrt{\frac{3}{\text{tr} \mathbf{A}}}(\mathbf{A} - \mathbf{I})\right). \end{aligned} \quad (2.35)$$

This leads to system of 9 non-linear equations that we must solve numerically for the 9 components of $\mathbf{A}$:

$$\begin{aligned} \dot{\gamma}(A_{xy} + A_{yx}) - \tau_d^{-1}(A_{xx} - 1) &= 2\tau_R^{-1} \left(1 - \sqrt{\frac{3}{\text{tr} \mathbf{A}}}\right) \left(A_{xx} + \beta \sqrt{\frac{3}{\text{tr} \mathbf{A}}}(A_{xx} - 1)\right), \\ \dot{\gamma}A_{yy} - \tau_d^{-1}A_{xy} &= 2\tau_R^{-1} \left(1 - \sqrt{\frac{3}{\text{tr} \mathbf{A}}}\right) \left(A_{xy} + \beta \sqrt{\frac{3}{\text{tr} \mathbf{A}}}A_{xy}\right), \end{aligned} \quad (2.37)$$

$$\dot{\gamma}A_{yz} - \tau_d^{-1}A_{xz} = 2\tau_R^{-1}\left(1 - \sqrt{\frac{3}{\text{tr}\mathbf{A}}}\right)\left(A_{xz} + \beta\sqrt{\frac{3}{\text{tr}\mathbf{A}}}A_{xz}\right), \quad (2.38)$$

$$\dot{\gamma}A_{yy} - \tau_d^{-1}A_{xy} = 2\tau_R^{-1}\left(1 - \sqrt{\frac{3}{\text{tr}\mathbf{A}}}\right)\left(A_{yx} + \beta\sqrt{\frac{3}{\text{tr}\mathbf{A}}}A_{yx}\right), \quad (2.39)$$

$$-\tau_d^{-1}(A_{yy} - 1) = 2\tau_R^{-1}\left(1 - \sqrt{\frac{3}{\text{tr}\mathbf{A}}}\right)\left(A_{yy} + \beta\sqrt{\frac{3}{\text{tr}\mathbf{A}}}(A_{yy} - 1)\right), \quad (2.40)$$

$$-\tau_d^{-1}A_{yz} = 2\tau_R^{-1}\left(1 - \sqrt{\frac{3}{\text{tr}\mathbf{A}}}\right)\left(A_{yz} + \beta\sqrt{\frac{3}{\text{tr}\mathbf{A}}}A_{yz}\right), \quad (2.41)$$

$$\dot{\gamma}A_{zy} - \tau_d^{-1}A_{zx} = 2\tau_R^{-1}\left(1 - \sqrt{\frac{3}{\text{tr}\mathbf{A}}}\right)\left(A_{zx} + \beta\sqrt{\frac{3}{\text{tr}\mathbf{A}}}A_{zx}\right), \quad (2.42)$$

$$-\tau_d^{-1}A_{zy} = 2\tau_R^{-1}\left(1 - \sqrt{\frac{3}{\text{tr}\mathbf{A}}}\right)\left(A_{zy} + \beta\sqrt{\frac{3}{\text{tr}\mathbf{A}}}A_{zy}\right), \quad (2.43)$$

$$-\tau_d^{-1}(A_{zz} - 1) = 2\tau_R^{-1}\left(1 - \sqrt{\frac{3}{\text{tr}\mathbf{A}}}\right)\left(A_{zz} + \beta\sqrt{\frac{3}{\text{tr}\mathbf{A}}}(A_{zz} - 1)\right). \quad (2.44)$$

To solve this system of equations, we will use scipy's 'fsolve' function. This solver uses a modified Newton's method and measures the error by how close the solved root is to the true root. The error is defined by

$$\text{error} = |y_{true} - y_k|$$

. The solver stops when the error reaches a certain tolerance level. Here, we have the allowed tolerance to $\varepsilon = 10^{-8}$, so we are confident that the solver is returning a correct answer. Furthermore, since it is a modified Newton's method solver, it converges to the solution quadratically [13]. That is,

$$\varepsilon_{k+1} = O(\varepsilon_k^2).$$

Because of this, we are confident that the solver is returning a correct value.

The deformation of the polymer with the Rouse term included is found in Figure 7. We can see that the polymer does not get as stretched as in the dumbbell case.

This is because there are two relaxation mechanisms acting to balance the shear flow.

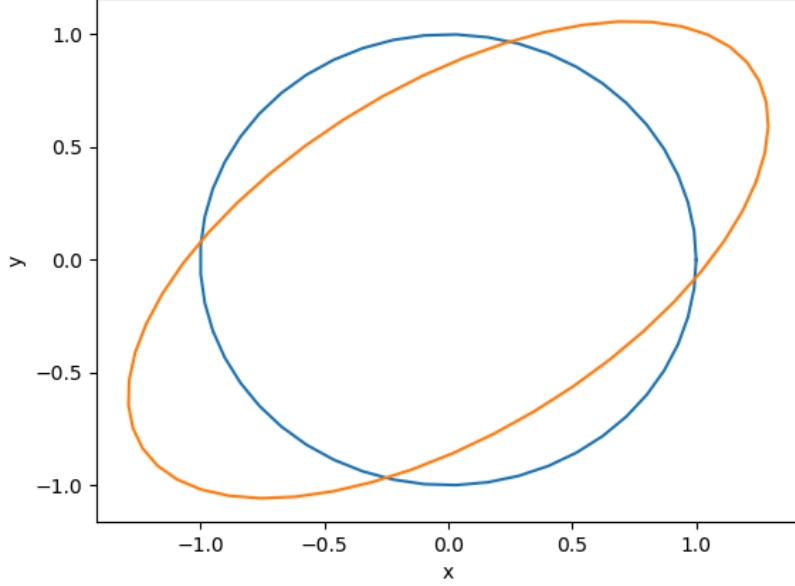


Figure 7: Deformation of a polymer satisfying the Rolie-Poly equation in a simple shear flow. Equilibrium state (blue) and deformation due to shear flow (orange)

If we use the values $\tau_r = 0.3, \tau_d = 5, \gamma = 5$ and $\beta = 1$ we get a steady state solution of

$$\mathbf{A} = \begin{pmatrix} 0.483304 & 0.898248 & 0 \\ 0.898248 & 3.8222 & 0 \\ 0 & 0 & 483304 \end{pmatrix}.$$

2.5.2 Start up behaviour

Now we solve the full Rolie-Poly equation in a shear flow numerically

$$\begin{aligned} \frac{d\mathbf{A}}{dt} = & \nabla \mathbf{u} \cdot \mathbf{A} + \mathbf{A} \cdot (\nabla \mathbf{u})^T + \frac{1}{\tau_d}(\mathbf{A} - \mathbf{I}) \\ & - \frac{2}{\tau_R} \left(1 - \sqrt{\frac{3}{\text{tr } \mathbf{A}}} \right) \left(\mathbf{A} + \beta \sqrt{\frac{3}{\text{tr } \mathbf{A}}}(\mathbf{A} - \mathbf{I}) \right). \end{aligned} \quad (2.45)$$

Similarly to before, this is just a set of 9 coupled differential equations that we will solve using a Runge-Kutta method. We will use scipy's 'solve_ivp' function. This is a Runge-Kutta 4(5) solver that has known error [14] of

$$\varepsilon = O(dt^4)$$

where dt is the set time step. When using this solver, our time step was set to $dt = 0.01$ so the error we have is $\varepsilon = 0.01^4 = 0.00000001$. Due to this low error, we are confident that the solver is returning a correct value.

We are only interested in the A_{xy} solution as it is the only one affected by the velocity gradient tensor. The solution can be found in Figure 8. We can see that the stress in the polymer builds very quickly as it is exposed to the strength of the shear flow. Then, the stress plateaus down and settles into a steady state. This plateauing happens because the polymer chains start relaxing by reptation. The reptation happens so late due to the magnitude of τ_d compared to τ_R . The system settles into a steady state when there is a balance between the stretching and relaxation in the polymer chains.

We can confirm we have the correct solution because it matches the steady state solution found in Section 2.5.1.

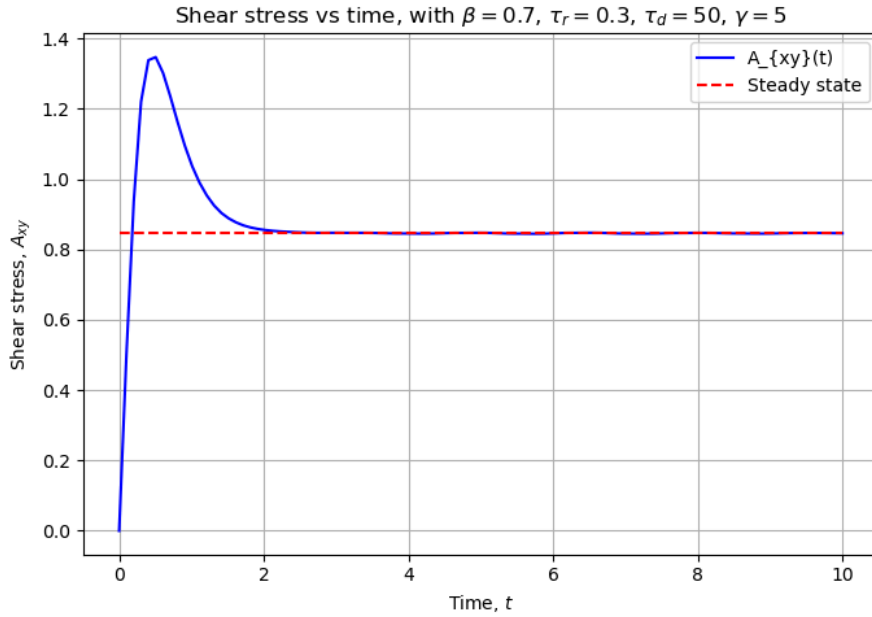


Figure 8: Plot of A_{xy} across for all time and a plot of the steady state solution.

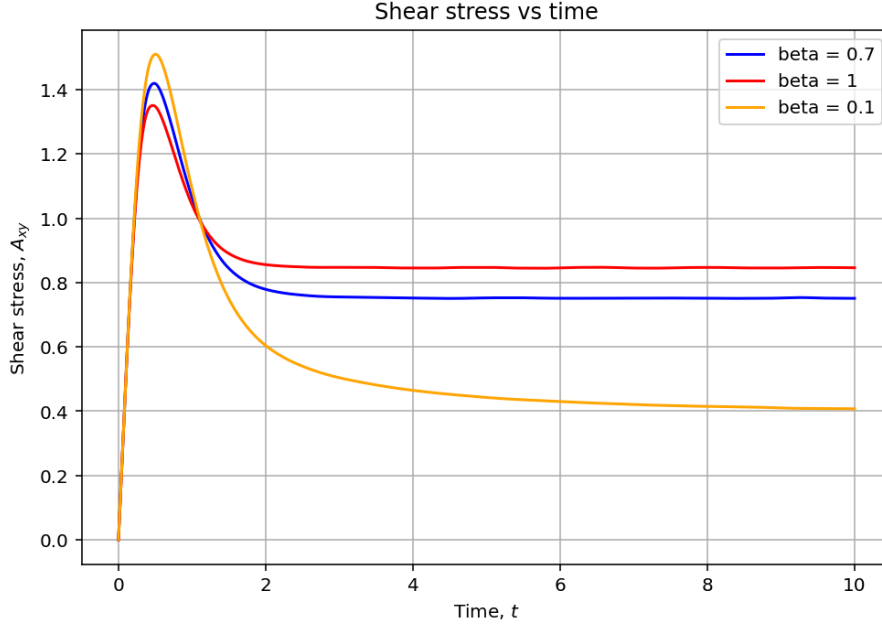


Figure 9: Plot of the stress for different CCR parameter values

We can also do some analysis on changing the CCR parameter. Figure 9 shows three solutions of Equation (2.33) for three different values of β . When β is small, the polymer takes longer to relax into steady state and the overshoot is larger. This is because there is less relaxation in the entanglements due to CCR and the relaxation agrees with a τ_r timescale. Conversely, when β is large ($\beta = 1$), the polymer gets into a steady state faster because the polymer chains relax faster due to more CCR.

We can also plot the stretch in the polymer, as found in Figure 10. Just as in the stress plot, we can see that the polymer stretches very quickly when it is first exposed to the shear flow. Then, the chains relax due to reptation after a short while (as the reptation time scale is longer). When the flow reaches steady state, we can see that $\Lambda > 1$, so there is always some stretching of the polymers in the steady state, which isn't apparent from the stress plot.

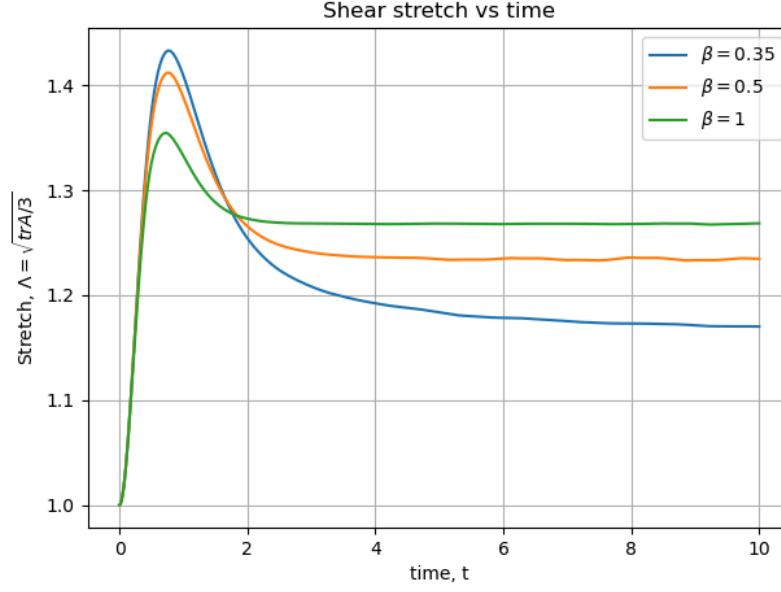


Figure 10: Plot of the stretch term across the rolie poly full time solution

We can also plot a constitutive plot to see how the behaviour of the polymer stress changes as the strength of the shear is increased. Figure 11 is a plot of the shear stress against increasing shear strengths for different values of β . We can see that when there is high CCR ($\beta = 1$), the shear stress is an increasing function of the shear strength. There is a slight decrease in the rate of stress growth where the polymer chains start relaxing. Conversely, when there is no CCR ($\beta = 0$), there are two turning points in the curve. The stress on the polymer chains decreases because of shear thinning. The polymers get stretched and since there is a lack of relaxation (from a lack of CCR), the melt gets less viscous and thus, needs less force to move. Then, when the chain starts relaxing due to reptation, they chains move more toward their equilibrium positions and more stress is needed to stretch them again.

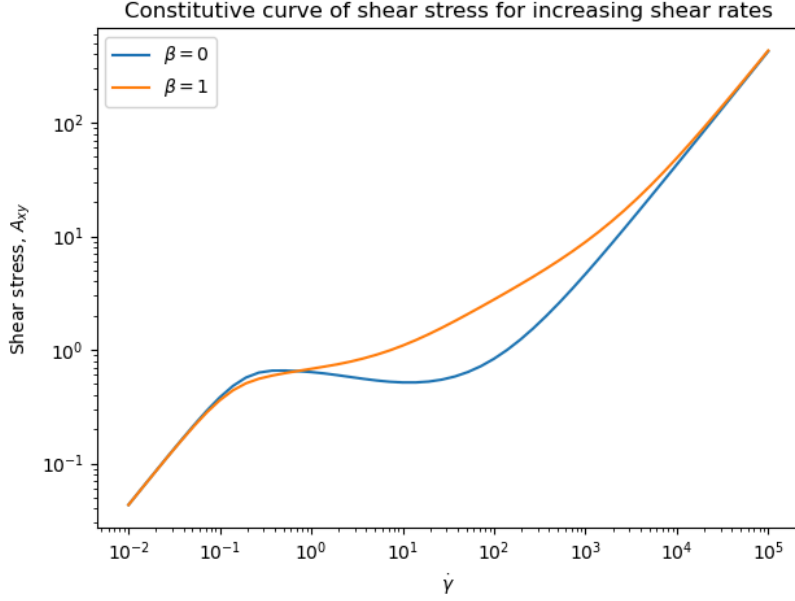


Figure 11: Constitutive curve of A_{xy} against $\dot{\gamma}$.

We can use this to further justify the results in Figure 9. The $\beta = 0$ curve takes longer to get to steady state because there is no CCR relaxation stopping the melt from thinning. The melt has to recover from the shear thinning effect before it gets into steady state.

When the polymer undergoes this shear thinning effect, the viscosity of the melt decreases, allowing the melt to flow easier. The viscosity in the polymer melt matches that of a Newtonian fluid until the shear rate reaches a critical value (as seen in Figure 12 and Figure 13). Then, the viscosity decreases which lets the fluid flow more easily. In the context of Figure 11, the viscosity starts decreasing when the shear stress decreases.

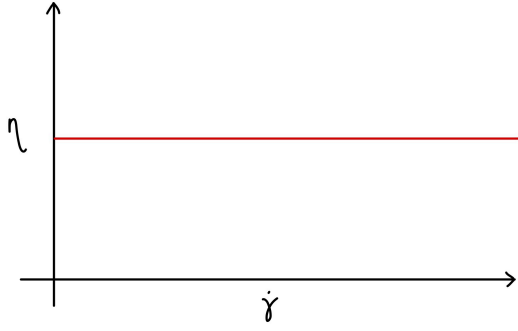


Figure 12: Newtonian fluid viscosity profile for increasing shear rates.

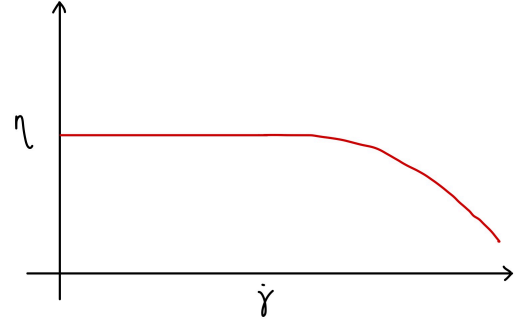


Figure 13: Non-Newtonian viscosity profile for increasing shear rates.

3 Modelling a polydisperse polymer melt

3.1 Polydisperse polymers

The Rolie-Poly equation describes the dynamics of a mono-disperse polymer melt. That is, a melt that is composed of polymer chains with no difference in molecular weight. However, polymer melts that are used in industry are typically polydisperse - they are composed of chains with a distribution of molecular weights. Figure 14 gives example molecular weight curves for two different polymer melts, a mono-disperse melt is shown by the dashed blue line.

Polydispersity is measured through gel permeation chromatography (GPC). This process involves passing a polymer melt through another porous material and measuring the lengths of polymer chains by the time it takes to get through the pores. Calculations of the molecular weight are made from these measurements, namely the number average molecular weight (M_n) and the weight average molecular weight (M_w) which are defined as

$$M_n = \frac{\sum_i N_i M_i}{\sum_i M_i}, \quad M_w = \frac{\sum_i N_i M_i^2}{\sum_i N_i M_i}.$$

Polydispersity is quantified by the polydispersity index. The dispersity of a polymer melt is defined by

$$\text{PDI} = \frac{M_w}{M_n}.$$

If a melt has a dispersity of 1 then it is mono-disperse and only contains chains of a single molecular weight. Table 1 shows some melts and their dispersity's.

Polymer melt	Use	Dispersity
Polystyrene standard	Packaging	1.0 - 1.5
Polybutadiene (industrial)	Rubber compounding	3
m-HDPE	Industrial Resins	12 - 42

Table 1: Polymers with different polydispersities and their uses. Values found from [15], [16], [17].

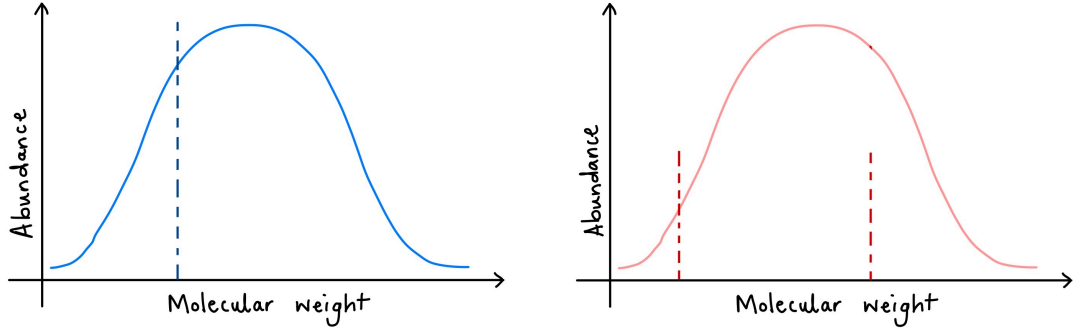


Figure 14: Molecular weight curves showing a mono-disperse melt (left) and a bi-disperse melt (right).

Much like the CCR mechanism in the Rolie-Poly model, there is another constraint release mechanism, double reptation, that must be included when more than one chain size is considered. Double reptation describes the new interaction between two different sized chains relaxing. The long chains and short chains are constrained by entanglements by each other however, since the short chains have a shorter relaxation time, when one reptates and untangles, it gives it's neighbouring chains more room to move. This lets the long chains relax more easily so it is effectively a new relaxation method.

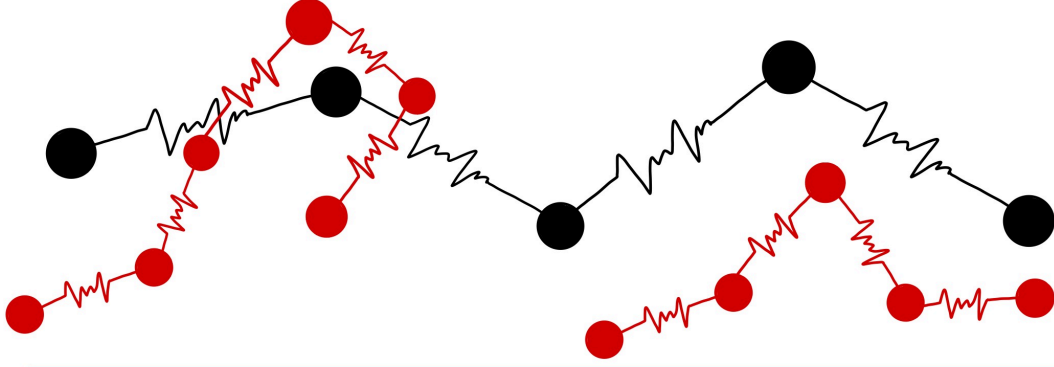


Figure 15: Diagram of slow reptation due to short and long chain entanglements in a tube (left) and faster reptation between unentangled chains (right).

In this section, we will simplify polydisperse polymer melts to a bi-disperse melt and consider the Rolie-Double-Poly model - an extension of the Rolie-Poly model that captures the dynamics of a bi-disperse melt. After setting up the model (following [3]), we will introduce a shear flow and do some analysis on the effects of changing the composition of the melt.

3.2 Rolie-Double-Poly model

There will be a proportion of the polymer melt, ϕ_S that is made from short chains and a proportion, ϕ_L made from long chains. As this will be a bi-modal model, there is a simple relationship for the melt composition,

$$\phi_S = 1 - \phi_L.$$

The polymer contribution to the stress tensor is now

$$\boldsymbol{\sigma} = G_e(\phi_S \mathbf{A}_S + \phi_L \mathbf{A}_L), \quad (3.46)$$

where $\mathbf{A}_S, \mathbf{A}_L$ are the deformation tensors for segments of short and long polymer chains. The long and short polymers will get entangled with each other, so they must exert a force on one another. For this reason, these deformation tensors take the forms

$$\begin{aligned} \mathbf{A}_S &= \phi_S \mathbf{A}_{SS} + \phi_L \mathbf{A}_{SL}, \\ \mathbf{A}_L &= \phi_L \mathbf{A}_{LL} + \phi_S \mathbf{A}_{LS}. \end{aligned} \quad (3.47)$$

These equations can be thought of being split into two sections, the deformation from purely short (or long) chains and then the deformation caused by the long chains on the short (or vice versa). $\mathbf{A}_{SS}$ and $\mathbf{A}_{LL}$ follow the single Rolie-Poly model (Equation (2.33)) and are defined as

$$\begin{aligned}\overset{\nabla}{\mathbf{A}}_{SS} &= \frac{1}{\tau_{d,S}}(\mathbf{A}_{SS} - \mathbf{I}) - \frac{2(1 - \lambda_S^{-1})}{\tau_{r,S}}(\mathbf{A}_{SS} + \beta\lambda_S(\mathbf{A}_{SS} - \mathbf{I})), \\ \overset{\nabla}{\mathbf{A}}_{LL} &= \frac{1}{\tau_{d,L}}(\mathbf{A}_{LL} - \mathbf{I}) - \frac{2(1 - \lambda_L^{-1})}{\tau_{r,L}}(\mathbf{A}_{LL} + \beta\lambda_L(\mathbf{A}_{LL} - \mathbf{I})).\end{aligned}\tag{3.48}$$

Here we have introduced the short hand

$$\overset{\nabla}{\mathbf{A}} \equiv \frac{d\mathbf{A}}{dt} - (\nabla \mathbf{u} \cdot \mathbf{A} + \mathbf{A} \cdot (\nabla \mathbf{u})^T).$$

The other deformation terms are defined as

$$\begin{aligned}\overset{\nabla}{\mathbf{A}}_{SL} &= \frac{1}{2\tau_{d,S}}(\mathbf{A}_{SL} - \mathbf{I}) - \frac{2(1 - \lambda_S)}{\tau_{s,S}}\mathbf{A}_{SL} - (\mathbf{A}_{SL} - \mathbf{I}) \left(\frac{\beta}{2\tau_{d,L}} + 2\beta_{CCR} \frac{1 - \lambda_L}{\tau_{s,L}} \lambda_S \right) \\ \overset{\nabla}{\mathbf{A}}_{LS} &= \frac{1}{2\tau_{d,L}}(\mathbf{A}_{LS} - \mathbf{I}) - \frac{2(1 - \lambda_L)}{\tau_{s,L}}\mathbf{A}_{LS} - (\mathbf{A}_{LS} - \mathbf{I}) \left(\frac{\beta}{2\tau_{d,S}} + 2\beta_{CCR} \frac{1 - \lambda_S}{\tau_{s,S}} \lambda_L \right).\end{aligned}\tag{3.49}$$

Similarly to the single mode Rolie-Poly, there is a stretching term,

$$\lambda_i = \sqrt{\frac{1}{3} \text{tr} \mathbf{A}_i}$$

for $i = L, S$. There is a second constraint release, β , term that accounts for the thermal energy release from short chains that drives the double reptation effect. In [3], these equations include a term, $f_E(\lambda)$, that encodes the extending dynamics of the polymer and it takes the form

$$f_E(\lambda) = \frac{1 - \lambda_{\max}}{1 - \lambda^2 \lambda_{\max}}.$$

For simplicity of the model, we will assume that this is equal to 1. Physically this assumption means that the polymer will have infinite extensibility.

We have used new relaxation times in these equations. τ_s is a stretch relaxation time and scales like

$$\tau_{sS} \sim M_s^2, \quad \tau_{sL} \sim M_L^2,$$

where Z is the molecular weight of the short (long) polymer sections. There is also a reptation time, τ_d , which accounts which scales like

$$\tau_{dS} \sim M_s^3, \quad \tau_{dL} \sim M_L^3.$$

3.3 Shear flow

We will subject the two polymer melt model to a shear flow and study the effect that changing the composition of the melt (more short or more long) has on the stress profile. Our model states that the polymers in the melt should have the shortest relaxation time when $\phi_S = 1$ and the melt is comprised of only short chains. In contrast, we expect to see the relaxation time rapidly increase as the chains in the melt get longer and longer. From Equation (3.47) and Equation (3.48), in the case where the melt is only long or short, we should see the exact same curves as Section 2.5.

A plot of the σ_{xy} component of the stress tensor can be found in Figure 16 and Figure 17. We can see that the time taken to get into steady state (and thus relaxation time) is longest when there are only long chains ($\phi_S = 0$) in the melt and shortest when there are only short chains ($\phi_S = 1$).

We can see that as the chains in the melt get longer, the amount of stress rapidly increases. The stress overshoot gets larger when the polymers chains are longer due to a longer relaxation time causing the stress-relaxation balance in the melt to occur at a later time. The rapid increase in the stress overshoot is also evidence of the double reptation effect we discussed earlier. More shorter chains speed up the reptation of the long chains.

We can also notice that the $\phi_S = 1$ and $\phi_S = 0$ cases match the single mode Rolie-Poly model we used earlier. Figure 12 shows the full time stress solution for a long chain and short chain single mode model. These plots are the same and serves as an indicator that the dual polymer model is a good model.

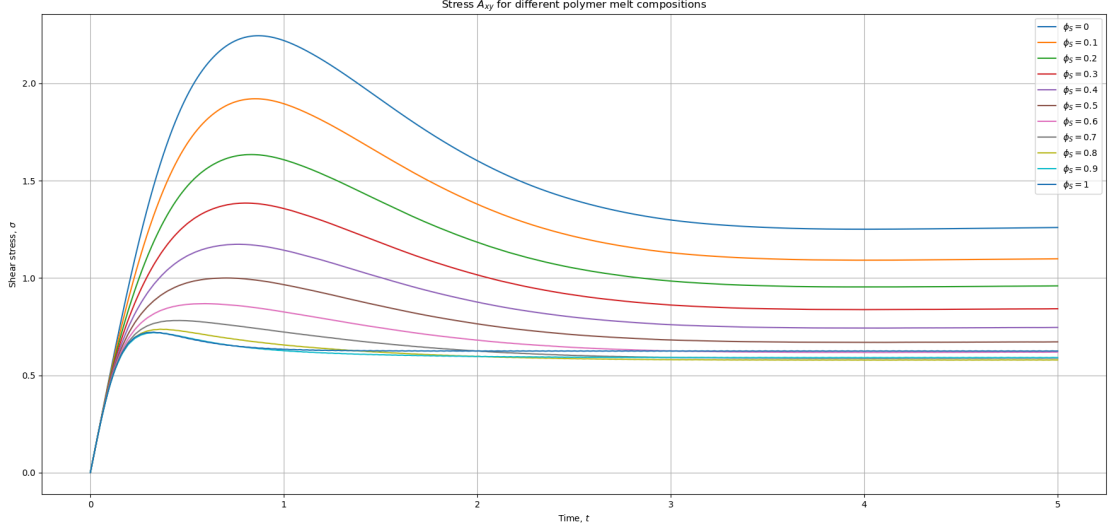


Figure 16: The A_{xy} component of the two polymer melt stress tensor in a shear flow, for increasing fractions of long chains.

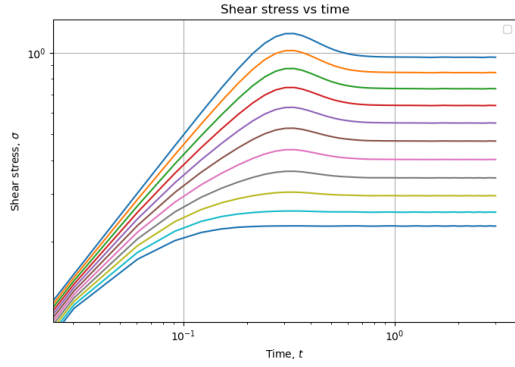


Figure 17: Logarithmic plot of the A_{xy} component of the two polymer shear flow stress.

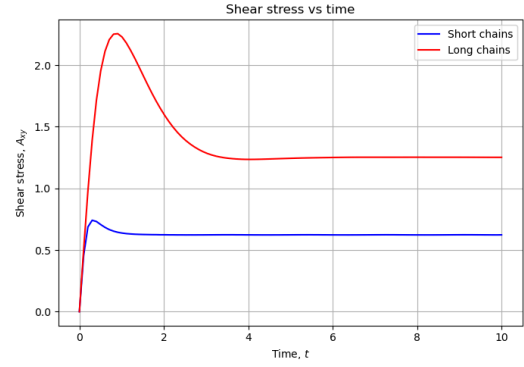


Figure 18: Full time solution of the single mode Rolie-Poly equation for short and long polymer chains.

3.4 Comparison to a monodisperse melt

We can plot the A_{xy}^S and A_{xy}^L components of stress for a single melt composition to understand how the two chains behave individually. Figure 19 shows the stress for a melt that is equal parts long and short chains ($\phi_S = \phi_L = 0.5$).

We can compare this stress to the long-short single mode solutions in Figure 16. The stress the short chains is higher in the 50-50 melt. This is because of entangling interactions (Equation (3.49)) between the long and short chains; more stress is needed to untangle the short chains from the longer ones. Conversely, the

stress in the long chains is lower in the 50-50 melt. The stress overshoot is also much lower in the 50-50 melt. This is more evidence of double reptation at work

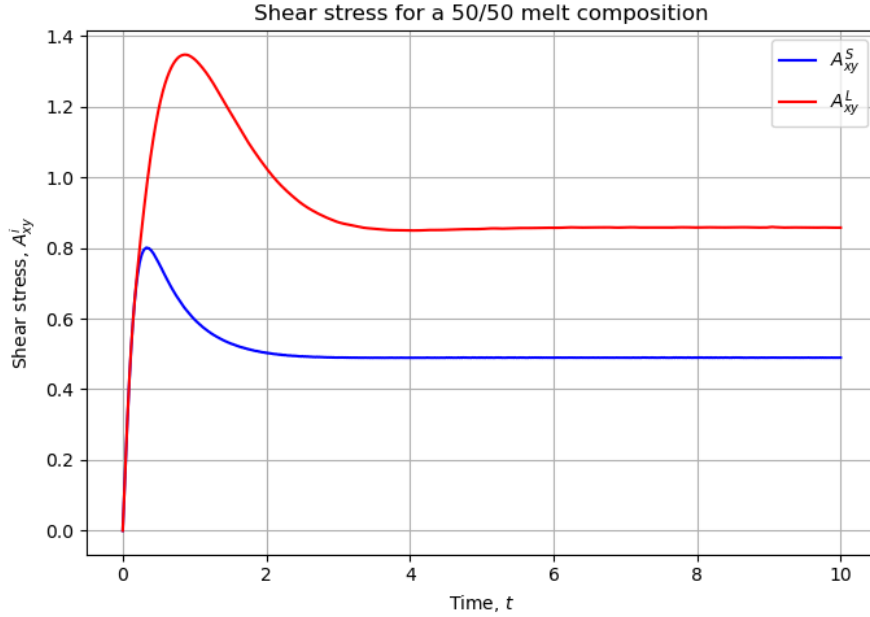


Figure 19: Shear stress, A_{xy} for a 50/50 long/short chain polymer melt

4 Viscous flow through a pipe

In this section we will discuss polymer flow in a pipe. We will first recall the well known flow profile for a Newtonian fluid, before subjecting the dumbbell and tube polymer models to a pipe flow.

4.1 Newtonian Flows

4.1.1 Basic steady state flow

Consider a steady-state, axisymmetric, incompressible and viscous flow down a circular pipe due to a pressure gradient.

For a flow profile $\mathbf{u} = (0, 0, w(r))$, the Navier-Stokes in cylindrical polars (r, ϕ, z) are given by

$$\begin{aligned}
 0 &= -\frac{1}{\rho} \frac{\partial p}{\partial r}, \\
 0 &= -\frac{1}{\rho} \frac{\partial p}{\partial z} + \frac{\mu}{r} \frac{\partial}{\partial r} \left(r \frac{\partial w}{\partial r} \right)
 \end{aligned} \tag{4.50}$$

The first equation states that p is independent of r but the second equation states that $\partial p / \partial z$ is a function of r . The only way this can be true is if $\partial p / \partial z$ is a constant, so we can set

$$p = p_0 - Gz.$$

Thus the problem to solve is simply the Navier-Stokes in the z direction with

$$0 = \frac{G}{\mu} + \frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial w}{\partial r} \right), \quad (4.51)$$

subject to the no slip condition

$$w = 0 \text{ on } r = a. \quad (4.52)$$

Integrating w.r.t r twice gives

$$\frac{dw}{dr} = -\frac{G}{2\mu}r + \frac{A}{r} \Rightarrow w = \frac{G}{4\mu}r^2 + A \ln r + B.$$

There are two constants. Letting $r \rightarrow 0$ gives an infinite solution due to the logarithmic term, so we must set $A = 0$. The second is found from the no-slip condition and gives

$$B = -\frac{G}{4\mu}a^2.$$

So the general solution is

$$w = -\frac{G}{4\mu}r^2 + \frac{G}{4\mu}a^2 = \frac{G}{4\mu}(a^2 - r^2). \quad (4.53)$$

If we plot this solution, as seen in Figure 20, we can see that the flow is a parabola centered at the middle of the cylinder with the flow slowing towards the sides and, of course, is stationary on the cylinder walls.

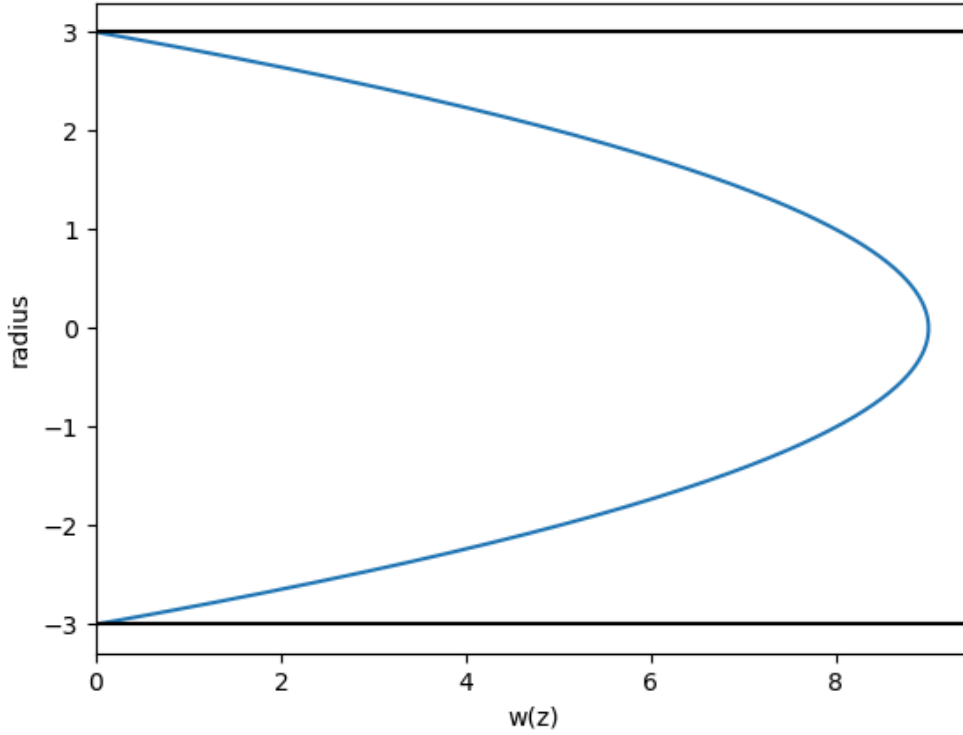


Figure 20: Plot of a fluid flow through a cylinder with $G = 2, a = 3, \mu = 0.5$. The blue line is the flow and the black are the walls of the cylinder.

4.1.2 Full time solution

If the flow is not in steady state, the system becomes

$$0 = -\frac{1}{\rho} \frac{\partial p}{\partial r}, \quad (4.54)$$

$$\frac{\partial w}{\partial t} = -\frac{1}{\rho} \frac{\partial p}{\partial z} + \frac{\mu}{r} \frac{\partial}{\partial r} \left(r \frac{\partial w}{\partial r} \right).$$

subject to the same boundary conditions as before, as well as the initial condition

$$w(r, 0) = 0.$$

This system doesn't have an analytical solution, so we must solve it numerically. We use MATLAB's 'pdepe' solver to solve this system, assuming a known pressure gradient. This is a parabolic and elliptical PDE solver that uses the numerical differential formulas [18]. The error is outlined in [19] as

$$\varepsilon = O(dt^5).$$

Similarly to the Runge-Kutta solver, this error gives us confidence that the solution we have is correct.

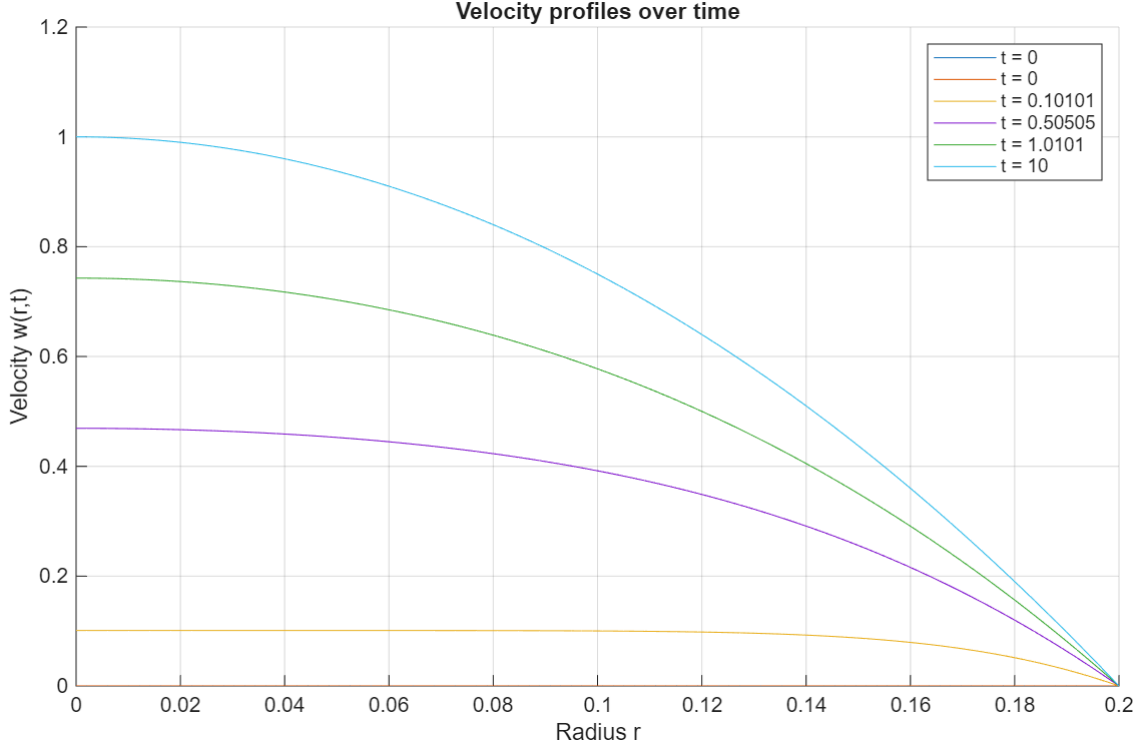


Figure 21 shows the evolution of the pipe flow over time. We see how the parabolic solution emerges as the flow approaches steady state.

4.2 Polymer Flows

As we have already seen in Section 2.1, polymer flow differs from Newtonian fluid flows in one aspect: there is a contribution from the polymer in the stress tensor. For this reason, to set up the problem we must start from the general form of the momentum equation. We have

$$\rho \frac{D\mathbf{u}}{Dt} = \nabla \cdot \boldsymbol{\sigma}. \quad (4.55)$$

Where $\frac{D}{Dt} = \left(\frac{\partial}{\partial t} + \mathbf{u} \cdot \nabla\right)$ is the material derivative,

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = \nabla \cdot \boldsymbol{\sigma}. \quad (4.56)$$

Now, the stress tensor takes the form of Equation (2.4),

$$\boldsymbol{\sigma} = p\mathbf{I} + 2\mu(\nabla \mathbf{u} + (\nabla \mathbf{u})^T) + G(\mathbf{A} - \mathbf{I}). \quad (4.57)$$

Substituting into Equation (4.56) gives,

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = \nabla p + 2\mu \nabla^2 \mathbf{u} + G(\nabla \cdot \mathbf{A}) \quad (4.58)$$

Again, we are dealing with pipe flow so the flow profile takes the form $\mathbf{u} = (0, 0, w(r))$ which gives the gradient velocity tensor

$$\nabla u = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ \frac{\partial w}{\partial r} & 0 & 0 \end{pmatrix}. \quad (4.59)$$

Substituting this into Equation (4.58) and breaking down into components gives the following system of equations

$$\begin{aligned} \frac{\partial p}{\partial r} + G \left(\frac{\partial A_{rr}}{\partial r} + \frac{1}{r} \frac{\partial A_{\phi r}}{\partial \phi} + \frac{\partial A_{zr}}{\partial z} + \frac{1}{r} (A_{rr} - A_{\phi\phi}) \right) &= 0 \\ \frac{1}{r} \frac{\partial p}{\partial \phi} + G \left(\frac{\partial A_{r\phi}}{\partial r} + \frac{1}{r} \frac{\partial A_{\phi\phi}}{\partial \phi} + \frac{\partial A_{z\phi}}{\partial z} + \frac{1}{r} (A_{r\phi} - A_{\phi r}) \right) &= 0 \\ \frac{\partial p}{\partial z} + 2\mu \left(\frac{\partial^2 w}{\partial r^2} + \frac{1}{r} \frac{\partial w}{\partial r} \right) + G \left(\frac{\partial A_{rz}}{\partial r} + \frac{1}{r} \frac{\partial A_{\phi z}}{\partial \phi} + \frac{\partial A_{zz}}{\partial z} + \frac{A_{rz}}{r} \right) &= \frac{\partial w}{\partial t}. \end{aligned} \quad (4.60)$$

These can be simplified greatly by assumption of the flow profile ($\frac{\partial}{\partial \phi} = \frac{\partial}{\partial z} = 0$),

$$\begin{aligned} \frac{\partial p}{\partial r} + G \left(\frac{\partial A_{rr}}{\partial r} + \frac{1}{r} (A_{rr} - A_{\phi\phi}) \right) &= 0 \\ \frac{1}{r} \frac{\partial p}{\partial \phi} + G \left(\frac{\partial A_{r\phi}}{\partial r} + \frac{1}{r} (A_{r\phi} - A_{\phi r}) \right) &= 0 \\ \frac{\partial p}{\partial z} + 2\mu \left(\frac{\partial^2 w}{\partial r^2} + \frac{1}{r} \frac{\partial w}{\partial r} \right) + G \left(\frac{\partial A_{rz}}{\partial r} + \frac{A_{rz}}{r} \right) &= \frac{\partial w}{\partial t}. \end{aligned} \quad (4.61)$$

As there is no flow in the r or ϕ direction, we only need to consider the z equation.

Thus, the system we are to solve is

$$\frac{\partial p}{\partial z} + 2\mu \left(\frac{\partial^2 w}{\partial r^2} + \frac{1}{r} \frac{\partial w}{\partial r} \right) + G \left(\frac{\partial A_{rz}}{\partial r} + \frac{A_{rz}}{r} \right) = \frac{\partial w}{\partial t}, \quad (4.62)$$

coupled with a deformation equation in $\mathbf{A}$, and subject to the boundary conditions:

$$w = 0 \text{ on } r = R \text{ and } \frac{\partial w}{\partial r} = 0 \text{ on } r = 0$$

as well as the initial conditions:

$$\mathbf{A}(t = 0) = \mathbf{I} \text{ and } w(t = 0) = 0$$

4.2.1 Dumbbell model in pipe

Let us subject the dumbbell model (Section 2.3) to the pipe flow we have derived.

Substituting the gradient velocity into Equation (2.23) gives

$$\begin{aligned} \frac{d\mathbf{A}}{dt} = & \begin{pmatrix} \frac{\partial w}{\partial r} A_{zr} & \frac{\partial w}{\partial r} A_{z\phi} & \frac{\partial w}{\partial r} A_{zz} \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} + \begin{pmatrix} \frac{\partial w}{\partial r} A_{zr} & 0 & 0 \\ \frac{\partial w}{\partial r} A_{\phi z} & 0 & 0 \\ \frac{\partial w}{\partial r} A_{zz} & 0 & 0 \end{pmatrix} \\ & - \frac{1}{\tau} \begin{pmatrix} A_{rr} - 1 & A_{r\phi} & A_{rz} \\ A_{\phi r} & A_{\phi\phi} - 1 & A_{\phi z} \\ A_{zr} & A_{z\phi} & A_{zz} - 1 \end{pmatrix} \end{aligned} \quad (4.63)$$

subject to the initial condition $\mathbf{A}(t = 0) = \mathbf{I}$. Similarly to Equation (2.23), the solutions to the components follow similar methods so we will only work through the A_{rz} component explicitly and then state the full solution. We have

$$\frac{dA_{rz}}{dt} = \frac{\partial w}{\partial r} - \frac{1}{\tau} A_{rz}, \quad A_{rz}(0) = 0 \quad (4.64)$$

Rearranging and introducing the integrating factor of $e^{t/\tau}$ yields

$$\begin{aligned} \frac{d}{dt}(e^{t/\tau} A_{rz}) &= \frac{\partial w}{\partial r} e^{t/\tau} \\ \Rightarrow A_{rz} &= \frac{\partial w}{\partial r} \tau + c e^{-t/\tau} \end{aligned} \quad (4.65)$$

Then imposing the initial condition gives

$$\begin{aligned} c &= -\frac{\partial w}{\partial r} \tau, \\ \Rightarrow A_{rz} &= \frac{\partial w}{\partial r} \tau - \frac{\partial w}{\partial r} \tau e^{-t/\tau}. \end{aligned} \quad (4.66)$$

The full solution of Equation (4.63) is

$$\mathbf{A} = \begin{pmatrix} A_{rr} & 0 & A_{rz} \\ 0 & 1 & 0 \\ A_{rz} & 0 & 1 \end{pmatrix} \quad (4.67)$$

with

$$A_{rr} = 1 + 2 \left(\frac{\partial w}{\partial r} \right)^2 \tau^2 (1 - e^{-t/\tau}) - 2 \left(\frac{\partial w}{\partial r} \right)^2 \tau t e^{-t/\tau},$$

$$A_{rz} = A_{zr} = \frac{\partial w}{\partial r} \tau (1 - e^{-t/\tau}).$$

We can now substitute into the flow equation (Equation (4.62)):

$$\frac{\partial w}{\partial t} = \frac{\partial p}{\partial z} + 2\mu \left(\frac{\partial^2 w}{\partial r^2} + \frac{1}{r} \frac{\partial w}{\partial r} \right) + G \left(\frac{\partial^2 w}{\partial r^2} + \frac{1}{r} \frac{\partial w}{\partial r} \right) \tau (1 - e^{-t/\tau}). \quad (4.68)$$

This is now an equation in w , and we can solve it numerically using the same method as in Section 4.1.2. A plot of the results can be found in Figure 22.

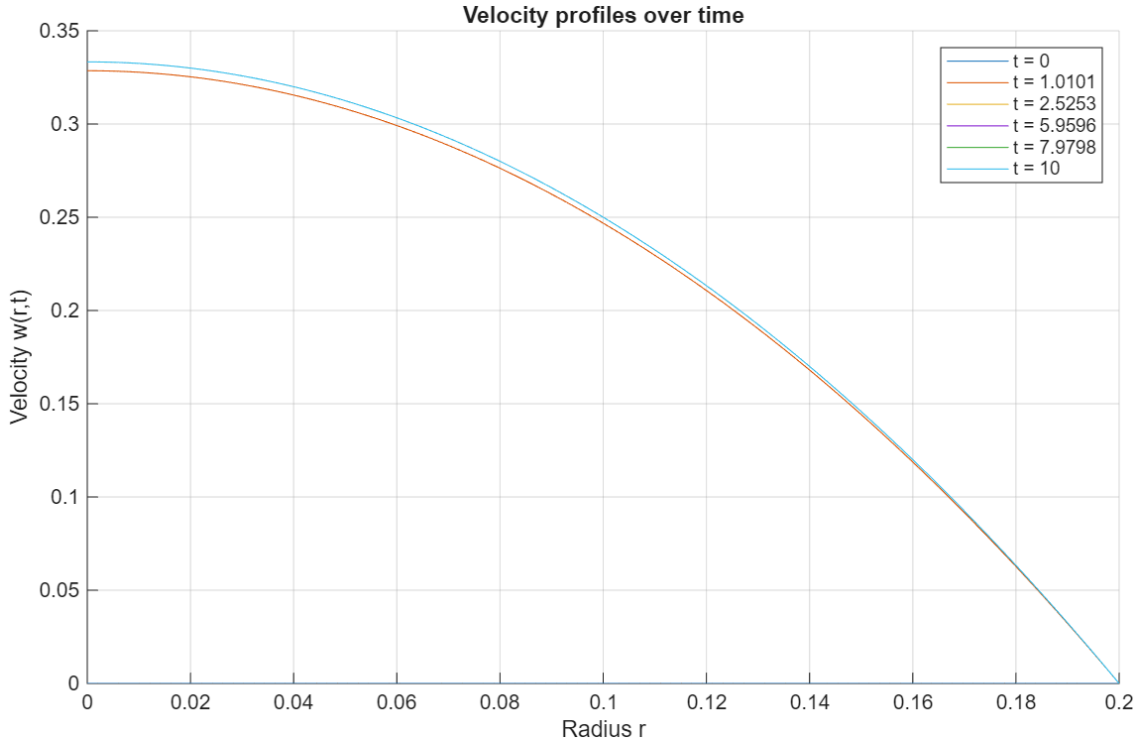


Figure 22: dumbbell model in pipe flow with $\mu = 0.01$, $\tau_d = 0.01$, $G = 2$, $dp/dz = 50$.

Figure 22 shows how the flow of the dumbbell evolves over time. We can see that the flow reaches steady state faster than the Newtonian fluid, reaching steady state at around $t = 2.5$.

4.2.2 Tube model in pipe flow

Now we will use subject the full Rolie-Poly model to a pipe flow. The system to solve is as follows:

$$\frac{\partial w}{\partial t} = \frac{\partial p}{\partial z} + 2\mu \left(\frac{\partial^2 w}{\partial r^2} + \frac{1}{r} \frac{\partial w}{\partial r} \right) + G \left(\frac{\partial A_{rz}}{\partial r} + \frac{A_{rz}}{r} \right) \quad (4.69)$$

coupled with the A_{rz} component of the Rolie-Poly equation,

$$\frac{dA_{rz}}{dt} = \frac{\partial w}{\partial r} A_{rr} - \frac{A_{rz}}{\tau_d} - \frac{2}{\tau_r} \left(1 - \sqrt{\frac{3}{\text{tr} \mathbf{A}}} \right) \left(A_{rz} - \beta A_{rz} \sqrt{\frac{3}{\text{tr} \mathbf{A}}} \right). \quad (4.70)$$

Unfortunately, unlike in the dumbbell flow, we cannot make much analytical progress in simplifying the system so we must solve this along with the equations for A_{rr} , $A_{\phi\phi}$ and A_{zz} :

$$\frac{dA_{rr}}{dt} = -\frac{1}{\tau_d} (A_{rr} - 1) - \frac{2}{\tau_r} \left(1 - \sqrt{\frac{3}{\text{tr} \mathbf{A}}} \right) \left(A_{rr} - \beta \sqrt{\frac{3}{\text{tr} \mathbf{A}}} (A_{rr} - 1) \right) \quad (4.71)$$

$$\frac{dA_{\phi\phi}}{dt} = -\frac{1}{\tau_d} (A_{\phi\phi} - 1) - \frac{2}{\tau_r} \left(1 - \sqrt{\frac{3}{\text{tr} \mathbf{A}}} \right) \left(A_{\phi\phi} - \beta \sqrt{\frac{3}{\text{tr} \mathbf{A}}} (A_{\phi\phi} - 1) \right) \quad (4.72)$$

$$\frac{dA_{zz}}{dt} = 2 \frac{\partial w}{\partial r} A_{rz} - \frac{1}{\tau_d} (A_{\phi\phi} - 1) - \frac{2}{\tau_r} \left(1 - \sqrt{\frac{3}{\text{tr} \mathbf{A}}} \right) \left(A_{\phi\phi} - \beta A_{\phi\phi} \sqrt{\frac{3}{\text{tr} \mathbf{A}}} \right) \quad (4.73)$$

Subject to the same boundary and initial conditions as before,

$$\mathbf{A}(0) = \mathbf{I}, \quad w(r, 0) = w(0, t) = \frac{\partial w(R, t)}{\partial t} = 0.$$

We will, again, solve this system using MATLAB's pdepe function but in order to do so we need to rearrange it into a form the solver can use. Equation (4.69) is the only equation we have to change and it must take the form

$$\frac{\partial w}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(2\mu \frac{\partial w}{\partial r} + G A_{rz} \right) + \frac{\partial p}{\partial z}.$$

The solution can be found in Figure 22. We can see an immediate different in the flow profile for the full Rolie-Poly flow compared to the dumbbell flow; the flow is blunt towards the centre of the pipe. Physically this represents a block of fluid in the middle of the pipe that flows together as a block, compared to a Newtonian fluid that has a velocity peak at $r = 0$. We can also see that the fluid flows a lot

faster than the Newtonian fluid, although this could be due to the values used when solving the equations. This block appears because of shear thinning within the melt.

Figure 24 shows the stress in the melt across the pipe. We can see that the stress gets stronger towards the walls of the pipe. At first glance the stress in the melt might not match the blunted velocity profile. However, upon close inspection the stress profile sharpens at around $r = \pm 0.1$.

To see the effect of this change in the melt, we plot the stretch term $\Lambda = \sqrt{3/\text{tr}\mathbf{A}}$, as seen in Figure 25. From this, we can see that the chains start getting stretched just before $r = \pm 0.1$ and get stretched more towards the walls of the pipe. Just as in the shear flow, when the polymer chains in the melt get stretched they are able to flow past each other more freely. This is the shear thinning effect we have already discussed.

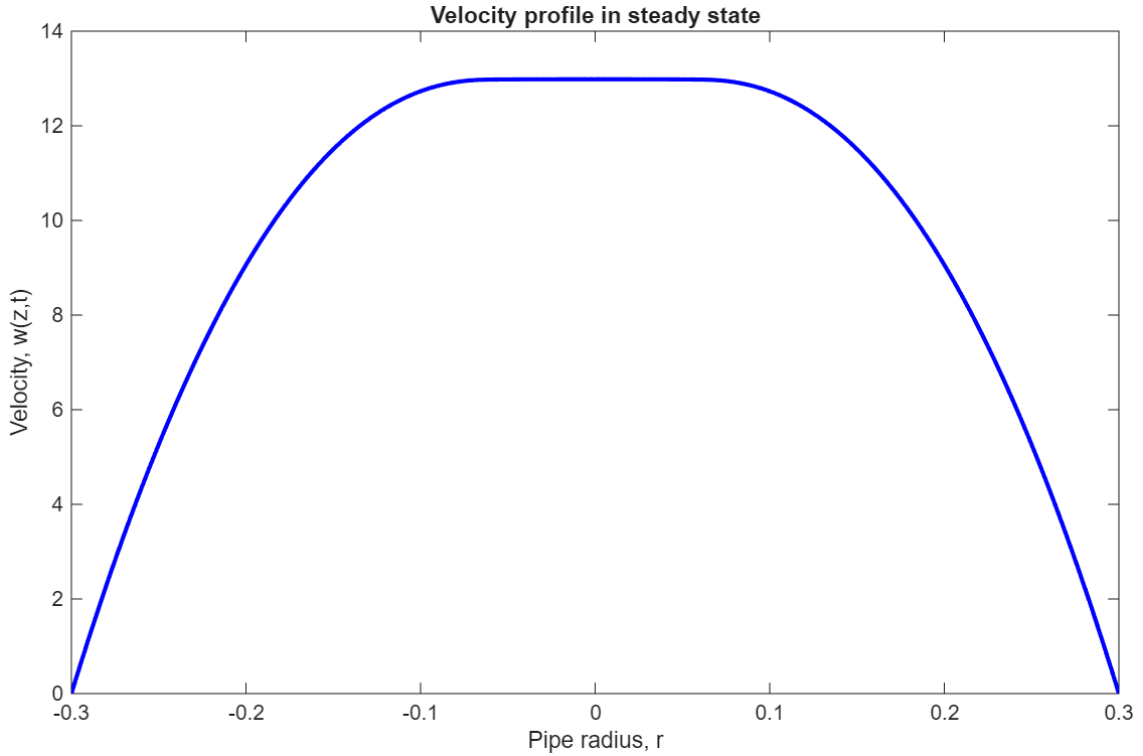


Figure 23: Solution for the full Rolie-Poly pipe flow with ...

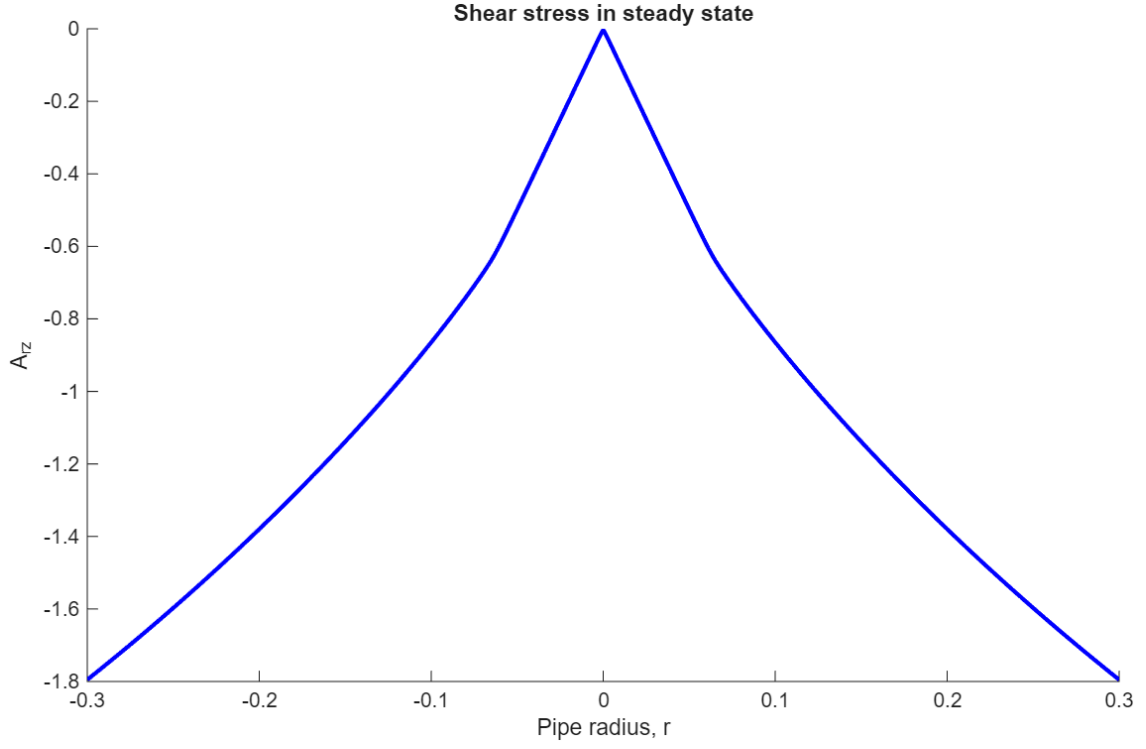


Figure 24: A_{rz} component of the deformation tensor for the full Rolie-Poly pipe flow.

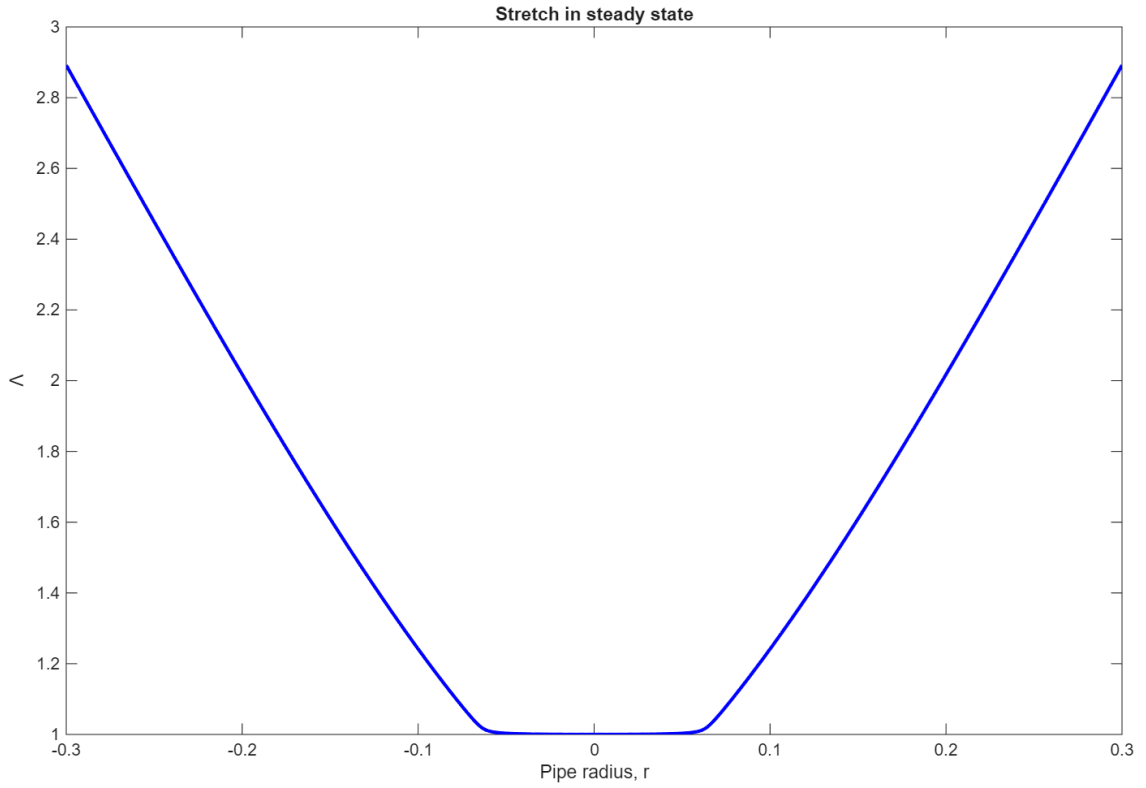


Figure 25: Stretch term $\Lambda = \sqrt{(\text{tr}\mathbf{A})/3}$ for full Rolie-Poly pipe flow

5 Conclusion

In conclusion, we have introduced various aspects of polymer modelling. We have briefly studied how modelling polymer chains geometrically can be used to derive equations of deformation for simple models like the dumbbell model. Furthermore, we have then seen the effects that different flows have on the deformation. Shear flows deformed the dumbbell to match the diagonal profile for a shear flow.

We have seen how to model chain entanglements in the tube model and by extending the tube theory to include extra relaxation mechanisms, we have seen how a shear flow causes deformation in entangled polymer chains. In particular, we have observed the stress overshoot effect that happens before the chains start relaxing. Importantly, we have seen how these shear polymer flows become steady when a stress/relaxation balance has been met. We observed the shear thinning effect that occurs when there is no relaxation from CCR in the model.

We extended the tube model theory further to a bi-disperse polymer melt and saw what the stress profile was like. Moreover, the stress profile in the long chains did not overshoot as much if there were more short chains in the melt composition.

Finally, we subjected both models to a pipe flow and compared the velocity profiles to that of a Newtonian fluid. The dumbbell model matched the Newtonian velocity profile but interestingly, We found that the Rolie-Poly model still had a shear thinning effect towards the walls of a pipe that caused a block of the melt to move together through the centre of the pipe.

Further work could include trying to model a full polydisperse polymer melt, instead a mono/bi disperse melt. Similarly, we could subject the bi-disperse model we have studied to a pipe flow and see the effects of changing the melt composition. Another avenue for further work is to introduce a temperature gradient into a flow. In this case, the viscosity is dependent on the temperature of the melt [20]. We could also investigate what happens if a melt is placed into a rotationally dominated flow.

6 Appendix

Here is the code used to plot the polymer deformation

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  plt.close('all')
5
6  domain = np.linspace(0,2*np.pi)
7  t=2
8
9
10 radius = 1
11 x0 = radius*np.cos(domain)
12 y0 = radius*np.sin(domain)
13 # z=0
14
15 tau_d = 0.2
16 gamma=5
17
18 A = [[1+gamma**2*tau_d**2, gamma*tau_d,0],
19      [gamma*tau_d, 1, 0],
20      [0,0,1]]
21
22 newx=[]
23 newy=[]
24
25 for i in domain:
26     x=radius*np.cos(i)
27     y=radius*np.sin(i)
28     z=1
29     pos=[x,y,z]
30     newpos = np.matmul(A,pos)
31     newx.append(newpos[0])
32     newy.append(newpos[1])
33
34 figure, axes = plt.subplots(1)
35 axes.plot(x0,y0)
36 axes.plot(newx,newy)
```

```
1  import numpy as np
2  import scipy as sc
```

```

3  import matplotlib.pyplot as plt
4
5  #we need to solve a linear of 9 equations for 9 unknowns.
6
7  def equations(A, AInitial=np.zeros(9)):
8
9      tau_d = 50
10     gamma = 5
11     beta = 0.5
12     tau_r = 3
13
14     A_xx, A_xy, A_xz, A_yx, A_yy, A_yz, A_zx, A_zy, A_zz = A
15
16     g = lambda A_xx, A_yy, A_zz: 2/tau_r * (1- np.sqrt(3/(A_xx+A_yy+A_zz)))
17     f = lambda A_xx, A_yy, A_zz: beta * np.sqrt(3/(A_xx+A_yy+A_zz))
18
19
20     eqset = [ gamma*(A_xy + A_yx) - 1/tau_d * (A_xx - 1) - g(A_xx,A_yy,A_zz) *
21               (A_xx + f(A_xx,A_yy,A_zz) * (A_xx -1)),
22               gamma * A_yy - 1/tau_d * A_xy - g(A_xx,A_yy,A_zz) * (A_xy +
23               f(A_xx,A_yy,A_zz)*A_xy),
24               gamma * A_yz - 1/tau_d * A_xz - g(A_xx,A_yy, A_zz) * (A_xz +
25               f(A_xx,A_yy,A_zz)*A_xz),
26               gamma * A_yy - 1/tau_d * A_yx - g(A_xx,A_yy, A_zz) * (A_yx +
27               f(A_xx,A_yy,A_zz) * A_yx),
28               - 1/tau_d * (A_yy -1) - g(A_xx,A_yy, A_zz) * (A_yy + f(A_xx,A_yy,A_zz) *
29               (A_yy -1)),
30               - 1/tau_d * A_yz - g(A_xx,A_yy, A_zz) * (A_yz + f(A_xx,A_yy,A_zz) *
31               A_yz),
32               gamma * A_zy - 1/tau_d * A_zx - g(A_xx,A_yy, A_zz) * (A_zx +
33               f(A_xx,A_yy,A_zz)*A_zx),
34               - 1/tau_d * A_zy - g(A_xx,A_yy, A_zz)*(A_zy + f(A_xx,A_yy,A_zz)*A_zy),
35               - 1/tau_d * (A_zz - 1) - g(A_xx,A_yy, A_zz) * (A_zz +
36               f(A_xx,A_yy,A_zz)*(A_zz-1))
37     ]
38
39     return eqset
40
41
42 A0 = np.ones(9)
43 AInitial = np.zeros(9)
44
45 domain = np.linspace(0,2*np.pi)

```

```

37 time = np.linspace(0,5,100)
38
39 A = sc.optimize.fsolve(equations, A0)
40
41 plt.close('all')
42
43 domain = np.linspace(0,2*np.pi)
44 t=2
45
46
47
48 radius = 1
49 x0 = radius*np.cos(domain)
50 y0 = radius*np.sin(domain)
51 # z=0
52
53 A = [ [A[0],A[1],A[2]],
54       [A[3],A[4],A[5]],
55       [A[6],A[7],A[8]]
56     ]
57
58
59 newx=[]
60 newy=[]
61
62
63 for i in domain:
64     x=radius*np.cos(i)
65     y=radius*np.sin(i)
66     z=1
67     pos=[x,y,z]
68     newpos = np.matmul(A,pos)
69     newx.append(newpos[0])
70     newy.append(newpos[1])
71
72 figure, axes = plt.subplots(1)
73 axes.plot(x0,y0)
74 axes.plot(newx,newy)

```

This is the code used to solve the Rolie-Poly equation:

```

1 import numpy as np
2 import scipy as sc

```

 Python

```

3  import matplotlib.pyplot as plt
4  from pdb import set_trace
5
6  plt.close('all')
7
8  #defining parameters
9  tau_r = 0.3
10 tau_d = 5
11 gamma = 5
12 beta = 3
13
14 #defining the velocity gradient for the flow
15 u = np.array([ [0,gamma,0], [0,0,0], [0,0,0] ])
16
17 #differential equation
18 def dAdt(A, t):
19
20     A = A.reshape(3, 3)
21     sqrt_term = np.sqrt(3 / np.trace(A))
22
23     dA = A @ u + u.T @ A + (1 / tau_d) * (A - np.identity(3)) \
24         - (2 / tau_r) * (1 - sqrt_term) * (A + beta * sqrt_term * (A - np.identity(3)))
25
26     return dA.flatten()
27
28 #steady state equation
29 def steadystate(A):
30
31     A = np.array(A)
32     A = A.reshape(3, 3)
33
34     sqrt_term = np.sqrt(3 / np.trace(A))
35
36     dA = A @ u + u.T @ A - (1 / tau_d) * (A - np.identity(3)) \
37         - (2 / tau_r) * (1 - sqrt_term) * (A + beta * sqrt_term * (A - np.identity(3)))
38
39     return dA.flatten()
40
41     A = np.array(A)
42     A = np.reshape(3,3)
43
44     sqrt = np.sqrt(3/np.trace(A))

```

```

45
46     dAdt = A @ u + u.T @ A - 1/tau_d * (A - np.identity(3)) - 2/tau_r * (1 - sqrt) * (A +
47     beta * sqrt) * (A-np.identity(3))
48
49     dAdt = dAdt.flatten()
50
51     return dAdt
52
53     #solving everything
54     t = np.linspace(0,10, 1000)
55     A0 = np.eye(3).flatten()
56     solution = sc.integrate.solve_ivp(lambda t, A: dAdt(A, t), [t[0], t[-1]], A0, t_eval=t).y
57
58     initial = solution[:, -1]
59     steadystatesolution = sc.optimize.fsolve(steadystate, initial)
60
61     #plotting everything
62     plt.figure(figsize=(7, 5))
63     plt.plot(t, solution[1,:], 'b', label='beta = 0.7')
64
65     # tau_r = 1
66     # tau_d = 100
67     beta=1
68     solution = sc.integrate.solve_ivp(lambda t, A: dAdt(A, t), [t[0], t[-1]], A0, t_eval=t).y
69     plt.plot(t, solution[1,:], 'r', label='beta = 1')
70
71     beta=0.4
72     solution = sc.integrate.solve_ivp(lambda t, A: dAdt(A, t), [t[0], t[-1]], A0, t_eval=t).y
73     plt.plot(t, solution[1,:], 'orange', label='beta = 0.1')
74     #plt.plot(t, steadystatesolution[1] * np.ones_like(t), 'r-', label='Steady state')
75     plt.xlabel(r'Time, $t$')
76     plt.ylabel(r'Shear stress, $A_{xy}$')
77     plt.title(r'Shear stress vs time')
78     plt.legend()
79     plt.grid(True)
80     plt.tight_layout()
81
82     plt.figure(figsize=(7,5))
83     for beta in [0,0.5,1]:
84         solution = sc.integrate.solve_ivp(lambda t, A: dAdt(A, t), [t[0], t[-1]], A0, t_eval=t).y
85         stretch = np.sqrt((solution[0,:]+solution[4,:]+solution[8,:])/3)

```

```

86 plt.plot(t, stretch, label=rf'$\beta = \{\beta\}$')
87 plt.legend()

```

This is the code to plot the constitutive curve:

```

1  import numpy as np
2  import scipy as sc
3  import matplotlib.pyplot as plt
4  from pdb import set_trace
5
6  #parameters
7  td = 5
8  tr = 0.3
9
10 #rolie poly eq
11 def RoliePoly(A,t):
12
13     u = np.array([ [0,gamma,0], [0,0,0], [0,0,0]])
14     A = np.array(A).reshape(3,3)
15     I = np.eye(3)
16     stretch = np.sqrt(3/np.trace(A))
17
18
19     dAdt = u @ A + A @ u.T - 1/td * (A - I) - 2/tr * (1-stretch)*(A + beta * stretch * (A -
20     I))
21     return dAdt.flatten()
22
23 #spaces and initial condition for solve_ivp
24 t = np.linspace(0,100,100)
25 A0=np.eye(3).flatten()
26 gammavals = np.logspace(-2,5,50)
27
28 plt.figure(figsize=(7, 5))
29 for beta in [0,1]:
30     curve=[]
31     for gamma in gammavals:
32         solution = sc.integrate.solve_ivp(lambda A,t: RoliePoly(t,A), [t[0], t[-1]], A0,
33         t_eval=t).y
34         a = solution[:,1][1]
35         curve.append(a)
36     plt.loglog(gammavals, curve, label=rf'$\beta = \{\beta\}$')

```

```

37
38 plt.xlabel(r'$\dot{\gamma}$')
39 plt.ylabel(r'Shear stress, $A_{xy}$')
40 plt.legend()

```

This is the code used to solve the Rolie-Double-Poly system:

```

1  import numpy as np
2  import scipy as sc
3  import matplotlib.pyplot as plt
4  from pdb import set_trace
5
6  plt.close('all')
7
8  #set parameters
9
10 tdS = 5
11 tsS = 0.01
12
13 tdL = 100
14 tsL = 1
15
16 Bth = 1
17 BCCR = 1
18
19 phi_S = 1
20 phi_L = 1 - phi_S
21
22 d = -0.5
23
24 G = 1
25
26 fE=1
27 #sets the flow gradient
28 gamma = 5
29 u = np.array([ [0,gamma,0], [0,0,0], [0,0,0] ])
30
31 I = np.identity(3)
32
33 #now define the equations to solve
34 def Ass(A,t):
35
36     A = A.reshape(3,3)

```

```

37     L = np.sqrt(1/3*np.trace(A))
38     #fE = (1-5**-2)/(1-L**2 * 5**-2)
39     dAssdt = u @ A + A @ u.T - 1/tdS * (A - I) - 2 * (1-L**-1)/tsS * fE * (A +
40     BCCR*L**(2*d)*(A - I))
41     return dAssdt.flatten()
42
43     def All(A,t):
44
45         A = A.reshape(3,3)
46         L = np.sqrt(1/3 * np.trace(A))
47         #fE = (1-5**-2)/(1-L**2 * 5**-2)
48         dAlldt = u @ A + A @ u.T - 1/tdL * (A - I) - 2 * (1-L**-1)/tsL * fE * (A +
49         BCCR*L**(2*d)*(A - I))
50         return dAlldt.flatten()
51
52     def Asl(A,t):
53
54         A = A.reshape(3,3)
55         L = np.sqrt(1/3 * np.trace(A))
56         #fE = (1-5**-2)/(1-L**2 * 5**-2)
57         dAsldt = u @ A + A @ u.T - 1/(2*tdS) * (A - I) - 2 * (1-L**-1)/tsS * fE * A - (A - I) *
58         (Bth/(2*tdL) + 2 * BCCR * (1-L**-1)/tsL * fE * L**(2*d))
59         return dAsldt.flatten()
60
61     def Als(A,t):
62
63         A = A.reshape(3,3)
64         L = np.sqrt(1/3 * np.trace(A))
65         #fE = (1-5**-2)/(1-L**2 * 5**-2)
66         dAsldt = u @ A + A @ u.T - 1/(2*tdL) * (A - I) - 2 * (1-L**-1)/tsL * fE * A - (A - I) *
67         (Bth/(2*tdS) + 2 * BCCR * (1-L**-1)/tsS * fE * L**(2*d))
68         return dAsldt.flatten()
69
70     #solves the stuff
71     t = np.linspace(0, 5, 10000)
72     A0 = np.eye(3).flatten()
73
74     def solve():

```

```

75
76     ss_solution = sc.integrate.solve_ivp(lambda t, A: Ass(A, t), [t[0], t[-1]], A0, t_eval=t).y
77     ll_solution = sc.integrate.solve_ivp(lambda t, A: All(A, t), [t[0], t[-1]], A0, t_eval=t).y
78
79     sl_solution = sc.integrate.solve_ivp(lambda t, A: Asl(A, t), [t[0], t[-1]], A0, t_eval=t).y
80     ls_solution = sc.integrate.solve_ivp(lambda t, A: Als(A, t), [t[0], t[-1]], A0, t_eval=t).y
81
82     #works out the total stress tensor
83     As = phi_S * ss_solution + phi_L * sl_solution
84     Al = phi_L * ll_solution + phi_S * ls_solution
85
86     sigma = G * (phi_S * fE * As + phi_L * fE * Al)
87
88     return [As, Al, sigma]
89
90     index = [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
91
92     plt.figure(figsize=(7, 5))
93     plt.title('Stress  $A_{xy}$  for different polymer melt compositions')
94     Assol=[]
95     Alsol=[]
96     #gets the solution for different phi values and plots
97     for i in index:
98         phi_S = i
99         phi_L = 1 - phi_S
100        sols = solve()[2][1]
101        Assol.append(solve()[0][1,:])
102        Alsol.append(solve()[1][1,:])
103        plt.plot(t, sols, label=f' $\phi_S = {i}$ ')
104
105
106     #plots the stuff
107     plt.xlabel(r'Time,  $t$ ')
108     plt.ylabel(r'Shear stress,  $\sigma$ ')
109     plt.legend()
110     plt.grid(True)
111     plt.tight_layout()
112
113
114
115     phi_L = 0.5
116     phi_S = 1 - phi_L

```

```

117
118
119 plt.figure(figsize=(7, 5))
120 plt.title('Shear stress for a 50/50 melt composition')
121 plt.plot(t,solve()[0][1,:], 'b', label='$A_{xy}^S$')
122 plt.plot(t,solve()[1][1,:], 'r', label='$A_{xy}^L$')
123 plt.legend()
124 plt.tight_layout()
125 plt.grid(True)
126
127
128 plt.figure(figsize=(7, 5))
129 plt.plot(t,Alsol[1])
130 plt.plot(t, Assol[1])
131
132 plt.xlabel(r'Time, $t$')
133 plt.ylabel(r'Shear stress, $A_s$')
134 plt.title(r'Shear stress vs time')
135 plt.legend()
136 plt.grid(True)
137 plt.tight_layout()
138
139 #shortStretch = np.sqrt((solve()[0][0,:]+solve()[0][4,:]+solve()[0][8,:])/3)
140 #longStretch = np.sqrt((solve()[1][0,:]+solve()[1][4,:]+solve()[1][8,:])/3)
141
142 plt.figure(figsize=(7,5))
143 for BCCR in [0,0.5,1,5]:
144     #set_trace()
145     longStretch = np.sqrt((solve()[1][0,:]+solve()[1][4,:]+solve()[1][8,:])/3)
146     plt.plot(t,longStretch, label=rf'$\beta = \{BCCR\}$')
147 plt.legend()
148
149 plt.figure(figsize=(7,5))
150 for BCCR in [0,0.5,1,5]:
151     #set_trace()
152     shortStretch = np.sqrt((solve()[0][0,:]+solve()[0][4,:]+solve()[0][8,:])/3)
153     plt.plot(t,shortStretch, '.', label=rf'$\beta = \{BCCR\}$')
154 plt.legend()

```

This is the code used to solve the Newtonian full time pipe flow:

```

1 function newtonian_pipe_flow
2     %parameters

```

Matlab

```

3    R = 0.2;
4    nu = 0.01;
5    dpdz = 1;
6    rho = 1;
7    tmax = 10;
8
9    %important stuff
10   r = linspace(0, R, 100);
11   t = linspace(0, tmax, 100);
12
13   %define the pde in the way the solver likes
14   function [c, f, s] = pdefun(r, t, w, dwdr)
15       c = 1;
16       f = nu * dwdr;
17       s = dpdz / rho;
18   end
19
20   %define the initial condition
21   function w0 = icfun(r)
22       w0 = 0;
23   end
24
25   %define the boundary conditions
26   function [pl, ql, pr, qr] = bcfun(rl, wl, rr, wr, t)
27       pl = 0; ql = 1; % dw/dr = 0 at r = 0
28       pr = wr; qr = 0; % w = 0 at r = R
29   end
30
31
32   %solve the stuff
33   m = 1;
34   sol = pdepe(m, @pdefun, @icfun, @bcfun, r, t);
35
36   %get the solution
37   w = sol(:, :, 1);
38
39   %plotting everything
40   figure;
41   hold on;
42   plot_times = [0.01, 0.05, 0.1, 0.5, 1.0, tmax]; % times to plot
43   colors = lines(length(plot_times));
44

```

```

45     for i = 1:length(plot_times)
46         [~, idx] = min(abs(t - plot_times(i)));
47         semilogx(r, w(idx, :), 'DisplayName', ['t = ' num2str(t(idx))], 'Color', colors(i,:));
48     end
49
50     xlabel('Radius r');
51     ylabel('Velocity w(r,t)');
52     title('Velocity profiles over time');
53     legend show;
54     grid on;
55 end

```

This is the code used to solve the dumbbell pipe flow:

```

1  function dumbbell_pipe_flow 🔗 Matlab
2      %parameters
3      R = 0.2;
4      mu = 0.01;
5      dpdz = 50;
6      rho = 1;
7      G=2;
8      td=0.01;
9      tmax = 10;
10
11
12     %important stuff
13     r = linspace(0, R, 100);
14     t = linspace(0, tmax, 1000);
15
16     %define the pde in the way the solver likes
17     function [c, f, s] = pdefun(r, t, w, dwdr)
18         c = 1;
19         f = (2*mu + G*td) * dwdr;
20         s = dpdz / rho + 1/r*dwdr*(G*td*(1-exp(-t/td))+2*mu);
21     end
22
23     %define the initial condition
24     function w0 = icfun(r)
25         w0 = 0;
26     end
27
28     %define the boundary conditions
29     function [pl, ql, pr, qr] = bcfun(rl, wl, rr, wr, t)

```

```

30     pl = 0; ql = 1; % dw/dr = 0 at r = 0
31     pr = wr; qr = 0; % w = 0 at r = R
32     end
33
34
35     %solve the stuff
36     m = 1;
37     sol = pdepe(m, @pdefun, @icfun, @bcfun, r, t);
38
39     %get the solution
40     w = sol(:,1);
41
42     %plotting everything
43     figure;
44     hold on;
45     plot_times = [0, 1, 2.5, 6, 8, tmax]; % times to plot
46     colors = lines(length(plot_times));
47
48     for i = 1:length(plot_times)
49         [~, idx] = min(abs(t - plot_times(i)));
50         plot(r, w(idx, :), 'DisplayName', ['t = ' num2str(t(idx))], 'Color', colors(i,:));
51     end
52
53     xlabel('Radius r');
54     ylabel('Velocity w(r,t)');
55     title('Velocity profiles over time');
56     legend show;
57     grid on;
58
59     rolie_poly_pipe_test
60 end

```

This is the code used to solve the Rolie-Poly pipe flow:

```

1  function roliepolypipeflow
2      %parameters
3      R = 0.3;
4      dpdz = 2000;
5      G = 100;
6      tr = 0.1;
7      beta = 1;
8      mu = 0.5;
9      td = 5;

```

Matlab

```

10     L = 100;
11     rho=1;
12     r = linspace(0, R, 300);
13     fullr = cat(2,-1*flip(r),r);
14     t = linspace(0, 50, 100);
15
16     %define the pde in the way the solver likes
17     function [c, f, s] = pdefun(r, t, w, dwdr)
18
19     g = sqrt(3/(w(3)+w(4)+w(5)));
20     c = [rho; 1; 1; 1; 1];
21
22     % Flux terms (f)
23     f1 = 2*mu*dwdr(1) + G*w(2);
24     f2 = 0;
25     f3 = 0;
26     f4 = 0;
27     f5 = 0;
28
29     f = [f1; f2; f3; f4; f5];
30
31     % Source terms (s)
32     s1 = dpdz;
33
34
35     A_rr = -1/td * (w(3)-1) - 2/tr*(1-g) * (w(3)+beta*g*(w(3)-1));
36     A_phiphi = -1/td * (w(4)-1) - 2/tr*(1-g) * (w(4)+beta*g*(w(4)-1));
37     A_rz = w(3) * dwdr(1) - 1/td * w(2) - 2/tr * (1-g)*(w(2) + g*beta*w(2));
38     A_zz = 2*dwdr(1) * w(2) - (w(5)-1)/td - 2/tr*(1-g) * (w(5)+beta*g*(w(5)-1));
39
40     s = [s1; A_rz; A_rr; A_phiphi; A_zz];
41
42     end
43
44     %define the initial condition
45     function w0 = icfun(r)
46     w0 = [0; 0; 1; 1; 1]; % [initial velocity; initial A_rz]
47     end
48
49     %define the boundary conditions
50     function [pl, ql, pr, qr] = bcfun(rl, ul, rr, ur, t)
51     % Left boundary: r = 0

```

```

52     pl = [0; 0; 0; 0; 0];
53     ql = [1; 1; 1; 1; 1];
54
55     % Right boundary: r = R (wall)
56     pr = [ur(1); 0; 0; 0; 0];
57     qr = [0; 1; 1; 1; 1];
58     end
59
60     %solve the stuff
61     m=1;
62     sol = pdepe(m, @pdefun, @icfun, @bcfun, r, t);
63
64
65
66     w = sol(:,:,1);    % velocity
67     w = cat(2,flip(w(end,:)),w(end,:));
68
69
70     Arz = sol(:,:,2);
71     Arz = cat(2,flip(Arz(end,:)),Arz(end,:));
72
73     Arr = sol(:,:,3);
74     Aphi = sol(:,:,4);
75     Azz = sol(:,:,5);
76
77
78     stretch = sqrt((Arr+Aphi+Azz)./3);
79     stretch = cat(2,flip(stretch(end,:)),stretch(end,:));
80
81     % Plot
82     figure;
83     plot(fullr, w, 'b', 'LineWidth', 2);
84     xlabel('Pipe radius, r'); ylabel('Velocity, w(z,t)');
85     title('Velocity profile in steady state');
86
87     figure;
88     hold on;
89     plot(fullr, Arz, 'b', 'LineWidth', 2);
90     xlabel('Pipe radius, r'); ylabel('A_{rz}');
91     title('Shear stress in steady state');
92
93     figure

```

```

94     plot(fullr, stretch, 'b', 'LineWidth',2)
95     xlabel('Pipe radius, r'); ylabel('trA');
96     title('Stretch in steady state');
97 end

```

Bibliography

- [1] M. Doi and S. F. Edwards, *The Theory of Polymer Dynamics*. Oxford University Press, 1986.
- [2] R. S. G. A. E. Likhtman, “Simple constitutive equation for liner polymer melts derived from molecular theory: Rolie-Poly equation.,” *J. Non-Newtonian Fluid Mech*, 2003.
- [3] V. A. H. Boudara, *Supramolecular and entangled polymer materials: rheological models*. University of Leeds, 2017.
- [4] J. W. Elliott, *Fluid Dynamics lecture notes*. University of Hull, 2023.
- [5] P. G. de Gennes, *Scaling Concepts in Polymer Physics*. Cornell University Press, 1979.
- [6] P. E. Rouse Jr, “A theory of the linear viscoelastic properties of dilute solutions of coiling polymers,” *The Journal of Chemical Physics*, vol. 21, no. 7, pp. 1272–1280, 1953.
- [7] R. A. O. H. R. Bird C. Curtiss, *Dynamics of Polymeric Liquids, Volume 2: Kinetic Theory*. Wiley-Interscience, 1987.
- [8] M. Fixman, “Radius of Gyration of Polymer Chains,” *The Journal of Chemical Physics*, 1962.
- [9] I. Newton and N. Chittenden, *Newton's Principia: the mathematical principles of natural philosophy*. Geo. P. Putnam, 1850.
- [10] G. Ianniruberto and G. Marrucci, “On compatibility of the Cox-Merz rule with the model of Doi and Edwards,” *Journal of Non-Newtonian Fluid Mechanics*, vol. 65, no. 2, pp. 241–246, 1996, [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0377025796014334>
- [11] L. G. L. D. J. R. Victor A.H. Boudara Joseph D. Peterson, “Nonlinear rheology of polydisperse blends of entangled linear polymers: Rolie-Double-Poly models.,” *Journal of Rheology*, 2019.
- [12] M. D. G.M. Mead R.G. Larson, “A molecular theory for fast flows of entangled polymers,” *Macromolecules* 31, 1998.

- [13] M. J. D. Powell, “A Hybrid Method for Nonlinear Equations,” *Numerical Methods for Nonlinear Algebraic Equations*. Gordon, Breach, pp. 87–114, 1970.
- [14] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Cengage Learning, 2010.
- [15] PolyAnalytik, “Conventional SEC/GPC Analysis.” Accessed: Aug. 20, 2025. [Online]. Available: <https://polyanalytik.com/resources/conventional-sec-gpc-analysis>
- [16] M. Mendez-Hernández, J. Rivera-Armenta, U. Paramo-Garcia, S. Corona, R. García-Alamilla, and B. Salazar Cruz, “Synthesis of High cis-1,4-BR with Neodymium for the Manufacture of Tires,” *International Journal of Polymer Science*, vol. 2016, p. , 2016, doi: 10.1155/2016/7239540.
- [17] M. Ansari, “Rheology and processing of high-density polyethylenes (HD-PEs): effects of molecular characteristics,” 2012.
- [18] MathWorks, Inc., “pdepe – Solve 1-D parabolic and elliptic partial differential equations.” 2025.
- [19] L. F. Shampine and M. W. Reichelt, “The MATLAB ODE Suite,” Natick, Massachusetts, technical report, 1997. [Online]. Available: https://www.mathworks.com/help/pdf_doc/otherdocs/ode_suite.pdf
- [20] J. K. & V. P. E. Závadský, “Non-isothermal flow of polymer melts in a curved tube,” *Rheologica Acta*, 1985.